

KIERUNEK: Automatyka i Robotyka

PRACA DYPLOMOWA
INŻYNIERSKA

Tytuł pracy:
Konstrukcja i budowa manipulatora
o sześciu stopniach swobody

AUTOR:
Michał Markuzel

PROMOTOR:
Dr inż. Michał Błędowski

Streszczenie

Celem pracy inżynierskiej jest zaprojektowanie i wykonanie modelu manipulatora robotycznego o sześciu stopniach swobody. Urządzenie zostało zaprojektowane w środowisku CAD, a następnie wykonane z wykorzystaniem technologii druku 3D. Praca obejmuje również cyfrową symulację oraz praktyczną implementację manipulatora wykonanego w skali.

Zaprojektowany model jest dobrym obiektem do analizy kinematyki manipulatorów, testowania algorytmów sterowania oraz integracji środowisk symulacyjnych z rzeczywistym modelem sprzętowym. Praca stanowi platformę umożliwiającą porównanie zachowania robota w środowisku symulacyjnym oraz w postaci fizycznej.

Zakres pracy obejmuje pełen proces inżynierski, poczynając od projektu mechanicznego manipulatora i przygotowania modeli CAD oraz plików do druku 3D, poprzez budowę jego cyfrowego odpowiednika w systemie ROS 2 wraz z wizualizacją i planowaniem ruchu, aż po implementację systemu sterowania na mikrokontrolerze ESP32 oraz opracowanie oprogramowania sterującego pracą przegubów manipulatora.

Efektem pracy jest kompletny model manipulatora, obejmujący zarówno konstrukcję mechaniczną, jak i warstwę programową. Opracowano modele 3D elementów robota, program sterujący mikrokontrolerem, opis kinematyczny robota w postaci modelu URDF oraz pakiet ROS 2 umożliwiający wizualizację i planowanie ruchu.

Wykonane oprogramowanie umożliwia sterowanie manipulatorem zarówno w środowisku symulacyjnym, jak i z wykorzystaniem rzeczywistego modelu sprzętowego, a także integrację z systemem ROS 2 w zakresie planowania trajektorii oraz wizualizacji ruchu. Praca dokumentuje kompletny cykl realizacji projektu – od koncepcji mechanicznej i analizy kinematycznej, poprzez przygotowanie modelu cyfrowego, aż po implementację systemu sterowania, integrację ze środowiskiem symulacyjnym oraz testy praktyczne.

Praca dokumentuje kompletny cykl realizacji projektu: od koncepcji mechanicznej, poprzez analizę kinematyczną i eksport cyfrowego modelu 3D, po implementację systemu sterowania, integrację ze symulacyjnym środowiskiem ROS 2 oraz testy praktyczne.

Słowa kluczowe: manipulator, CAD, druk 3D, kinematyka, system sterowania, ROS 2, URDF, MoveIt 2, planowanie ruchu.

Abstract

The objective of this engineering thesis is the design and construction of a robotic manipulator model with six degrees of freedom. The device was designed in a CAD environment and subsequently manufactured using 3D printing technology. The thesis also includes digital simulation and practical implementation of the scaled manipulator.

The designed model constitutes a suitable platform for the analysis of manipulator kinematics, testing of control algorithms, and integration of simulation environments with a real hardware model. The thesis provides a framework enabling comparison of the robot's behavior in a simulation environment and in its physical form.

The scope of the thesis covers the complete engineering process, starting from the mechanical design of the manipulator and preparation of CAD models and files for 3D printing, through the development of its digital counterpart in the ROS 2 environment with visualization and motion planning, to the implementation of the control system on an ESP32 microcontroller and the development of software responsible for manipulator joint control.

The result of the thesis is a complete manipulator model, comprising both the mechanical structure and the software layer. The work includes the development of 3D models of the robot components, a microcontroller control program, a kinematic description of the robot in the form of a URDF model, and a ROS 2 package enabling visualization and motion planning.

The developed software enables control of the manipulator both in a simulation environment and using the real hardware model, as well as integration with the ROS 2 system in the areas of trajectory planning and motion visualization. The thesis documents the complete project realization process—from the mechanical concept and kinematic analysis, through preparation of the digital model, to the implementation of the control system, integration with the simulation environment, and practical testing.

The thesis documents the complete project lifecycle: from the mechanical concept, through kinematic analysis and export of the digital 3D model, to the implementation of the control system, integration with the ROS 2 simulation environment, and practical tests.

Keywords: manipulator, CAD, 3D printing, kinematics, control system, ROS 2, URDF, MoveIt 2, motion planning.

Spis treści

1	Wstęp	1
1.1	Sformułowanie problemu	2
1.2	Cele pracy	2
1.3	Struktura pracy	2
2	Budowa manipulatora	5
2.1	Założenia	5
2.1.1	Rozwiązanie nr 1 - silniki krokowe	5
2.1.2	Rozwiązanie nr 2 - serwomechanizmy	6
2.1.3	Rozwiązanie nr 3 - hybrydowe	7
2.1.4	Ostateczny wybór	8
2.2	Opis budowy	9
2.2.1	Człon nr 0 - podstawa	9
2.2.2	Człon nr 1	10
2.2.3	Człon nr 2	11
2.2.4	Człon nr 3	12
2.2.5	Człon nr 4	13
2.2.6	Człon nr 5	14
2.2.7	Człon nr 6 – efektor	15
2.3	Złożenie całkowite	16
2.4	Układ sterujący	17
2.5	Rzeczywisty model manipulatora	18
3	Analiza kinematyki manipulatora	19
3.1	Schemat Kinematyczny	19
3.2	Tabela parametrów Denavita–Hartenberga	19
3.3	Macierze Transformacji	20
3.3.1	Ogólna macierz transformacji A_{i-1}^i	20
3.3.2	Macierze transformacji dla manipulatora	22
3.4	Transformacja całkowita T_0^9	22
4	Eksport modelu	25
4.1	Format URDF/XACRO	25
4.2	Proces przygotowania modelu	27
4.3	Konfiguracja przegubów i układów współrzędnych	27
4.4	Eksport przy użyciu skryptu	28

4.5	Uruchomienie procedury eksportu	29
4.6	Struktura wygenerowanego pakietu ROS 2	30
4.7	Weryfikacja modelu w środowisku RViz	31
4.8	Podsumowanie	32
5	Tworzenie pakietu MoveIt 2	33
5.1	Wprowadzenie do platformy MoveIt 2	33
5.2	Optymalizacja detekcji kolizji	34
5.3	Definiowanie grup planowania	35
5.4	Konfiguracja kontrolerów	36
5.5	Predefiniowane pozy robota	37
5.6	Konfiguracja kontrolerów ROS 2	38
5.7	Generowanie pakietu konfiguracyjnego	39
5.8	Podsumowanie konfiguracji	39
6	Funkcjonalności pakietu MoveIt2	41
6.1	Możliwości kinematyczne manipulatora	41
6.2	Konwencje wizualizacji w środowisku MoveIt	41
6.3	Planowanie i weryfikacja trajektorii	42
6.4	Detekcja kolizji	43
6.5	Analiza pośrednich konfiguracji trajektorii	43
6.6	Podsumowanie	46
7	Sterownik manipulatora	47
7.1	Architektura systemu sterowania	47
7.2	Główna pętla sterowania	47
7.3	Komunikacja szeregową i wykonywanie poleceń	49
7.4	Sterowanie serwomechanizmami	49
8	Integracja sterownika z ROS 2	51
8.1	Przepływ danych w systemie	51
8.2	Specyfikacja węzła komunikacyjnego	51
8.3	Opis algorytmu	52
8.4	Metodologia testowania	54
8.5	Testy realizacji ruchu	54
9	Wnioski i prace przyszłe	59
9.1	Podsumowanie	59
9.2	Ograniczenia zrealizowanego systemu	59
9.3	Kierunki dalszego rozwoju	60

1. Wstęp

W ostatnich latach można zaobserwować dynamiczny rozwój technologii oraz elektroniki, który znacząco wpływa na sposób projektowania i wytwarzania współczesnych urządzeń. Sprzęt, z którego korzystamy na co dzień, staje się coraz bardziej zaawansowany pod względem funkcjonalnym i konstrukcyjnym, a procesy jego produkcji są wielokrotnie bardziej złożone niż w przeszłości. Wzrost stopnia skomplikowania wyrobów idzie w parze ze stale rosnącym zapotrzebowaniem rynkowym, co zmusza przedsiębiorstwa do ciągłej optymalizacji i automatyzacji procesów produkcyjnych.

Jednym z kluczowych elementów nowoczesnych linii produkcyjnych jest automatyzacja zadań powtarzalnych, wymagających wysokiej precyzji, niezawodności oraz dużej wydajności. W tym celu szerokie zastosowanie znajdują różnego rodzaju roboty przemysłowe oraz maszyny automatyczne. Przykładami takich zastosowań są operacje montażowe, spawanie, paletyzacja, pakowanie, malowanie, obsługa obrabiarek CNC czy sortowanie elementów [1, 2]. W wielu z tych procesów praca wykonywana przez człowieka zostałaby ograniczona przez zmęczenie, niską powtarzalność lub warunki niebezpieczne, natomiast roboty umożliwiają ich realizację w sposób ciągły i stabilny.

Szczególną grupę robotów przemysłowych stanowią manipulatory, które dzięki swojej budowie umożliwiają przemieszczanie oraz orientowanie narzędzia roboczego w przestrzeni roboczej. W zależności od konstrukcji wyróżnia się m.in. manipulatory kartezjańskie, cylindryczne, sferyczne, typu SCARA oraz manipulatory przegubowe [3]. Manipulatory przegubowe cechują się największą elastycznością ruchu ze względu na wieloosiową strukturę kinematyczną opartą na przegubach obrotowych, która umożliwia niezależne sterowanie położeniem i orientacją efektora końcowego w przestrzeni trójwymiarowej. Właściwość ta jest szeroko wykorzystywana w robotach przemysłowych takich jak manipulatory firm KUKA, ABB czy FANUC, stosowane w zadaniach wymagających realizacji złożonych trajektorii i pracy w ograniczonej przestrzeni roboczej [4].

Istotną rolę w automatyzacji procesów produkcyjnych odgrywają manipulatory o sześciu stopniach swobody (6-DOF), które swoją strukturą kinematyczną przypominają ludzkie ramię. Tego rodzaju manipulatory umożliwiają pełne sterowanie położeniem oraz orientacją efektora w przestrzeni trójwymiarowej. Dzięki temu mogą realizować złożone trajektorie ruchu oraz wykonywać skomplikowane operacje technologiczne w ograniczonej przestrzeni roboczej. Manipulatory charakteryzują się również dużą uniwersalnością, co pozwala na ich zastosowanie w wielu gałęziach przemysłu.

Przykładami powszechnie stosowanych manipulatorów sześciostopniowych są roboty przemysłowe firm takich jak KUKA [5], ABB [6], FANUC [7] czy Universal Robots [8], wykorzystywane m.in. w przemyśle motoryzacyjnym, elektronicznym oraz w zautomatyzowanych stanowiskach montażowych. Ich popularność wynika z połączenia wysokiej precyzji, powtarzalności oraz możliwości łatwej adaptacji do różnych zadań produkcyjnych.

1.1. Sformułowanie problemu

W pracy przedstawiono opracowanie manipulatora robotycznego o sześciu stopniach swobody, umożliwiającego zarówno symulację ruchów w środowisku wirtualnym, jak i ich realizację na modelu fizycznym. Istotnym elementem projektu było przygotowanie cyfrowego modelu manipulatora wraz z opisem kinematycznym, pozwalającym na wizualizację oraz analizę ruchu. Praca obejmuje również implementację sterowania manipulatorem z wykorzystaniem mikrokontrolera oraz jego integrację z systemem symulacyjnym. Takie podejście umożliwia bezpośrednią weryfikację poprawności konstrukcji oraz porównanie zachowania modelu symulacyjnego z rzeczywistym układem sprzętowym.

1.2. Cele pracy

Celem pracy inżynierskiej jest zaprojektowanie oraz wykonanie modelu przemysłowego manipulatora robotycznego o sześciu stopniach swobody w skali. Manipulator ma być zaprojektowany w środowisku CAD, a jego elementy mechaniczne wykonane z wykorzystaniem technologii druku 3D.

1.3. Struktura pracy

Praca została podzielona na **dziewięć rozdziałów**, z których każdy omawia kolejny etap projektowania, analizy, implementacji oraz integracji sześciostopniowego manipulatora:

1. **Wstęp** — przedstawienie problemu badawczego, celów pracy oraz zakresu realizowanego projektu.
2. **Budowa manipulatora** — omówienie założeń projektowych, porównanie wariantów napędu, wybór finalnej koncepcji, spis materiałów oraz szczegółowy opis poszczególnych elementów konstrukcyjnych wraz z pełnym złożeniem manipulatora i układem sterującym.
3. **Analiza kinematyki manipulatora** — przygotowanie tabeli parametrów Denavit–Hartenberga (DH), schematu kinematycznego, wyprowadzenie macierzy transformacji oraz pełnej kinematyki prostej.
4. **Eksport modelu** — wprowadzenie do formatu URDF/XACRO, proces przygotowania i konfiguracji modelu, ustawienie stawów i układów współrzędnych, eksport przy użyciu skryptu, analiza struktury wygenerowanego pakietu oraz weryfikacja działania modelu w symulacji RViz.
5. **Tworzenie pakietu MoveIt 2** — omówienie platformy MoveIt 2, konfiguracja kolizji, kontrolerów wykonawczych, grup planowania, predefiniowanych pozycji robota oraz kontrolerów ROS 2, a także generacja kompletnego pakietu konfiguracyjnego.

6. **Funkcjonalności pakietu MoveIt 2** — opis możliwości kinematycznych manipulatora, standardów wizualizacji, planowania i weryfikacji trajektorii, detekcji kolizji oraz analizy konfiguracji pośrednich.
7. **Sterownik manipulatora** — architektura systemu sterowania, struktura projektu, główna pętla sterowania, komunikacja UART, hybrydowe sterowanie serwomechanizmami, konfiguracja parametrów oraz integracja z ROS 2.
8. **Integracja sterownika z ROS 2** — połączenie części embedded z systemem ROS 2, testowanie komunikacji, sprawdzenie poprawności generowanych trajektorii oraz ocena pracy manipulatora w warunkach rzeczywistych.
9. **Wnioski i prace przyszłe** — podsumowanie wyników, identyfikacja mocnych i słabych stron rozwiązania, wskazanie możliwych kierunków rozwoju systemu.

2. Budowa manipulatora

2.1. Założenia

Projekt mechaniczny manipulatora obejmował szereg istotnych aspektów konstrukcyjnych. Etap projektowania rozpoczęto od wyboru odpowiedniego rozwiązania sterowania, obejmującego przede wszystkim określenie skali urządzenia. Ze względu na założony zakres zastosowań zdecydowano o budowie manipulatora o wysokości około 500 mm, co podyktowało wybór odpowiedniego systemu napędu [9].

Rozpatrzono trzy koncepcje realizacyjne:

1. Napędzanie wszystkich przegubów silnikami krokowymi z przekładniami sferycznymi lub cykloidalnymi, wyposażonymi w pętlę sprzężenia zwrotnego z enkoderów.
2. Napędzanie wszystkich przegubów serwomechanizmami bez pętli sprzężenia zwrotnego.
3. Hybrydowe rozwiązanie: napęd przegubów 0 i 1 silnikami krokowymi z transmisją pasową, przeguby 2-5 sterowane serwomechanizmami.

Każde z wymienionych rozwiązań charakteryzowało się pewnymi zaletami i ograniczeniami, omówionymi szczegółowo w dalszych podrozdziałach.

2.1.1. Rozwiązanie nr 1 - silniki krokowe

Rozwiązanie oparte na zastosowaniu silników krokowych jest powszechnie spotykane zarówno w profesjonalnych konstrukcjach przemysłowych, jak i w zaawansowanych projektach edukacyjnych oraz prototypowych. Silniki te są często wykorzystywane w aplikacjach wymagających precyzyjnego pozycjonowania oraz możliwości generowania wysokich momentów obrotowych, szczególnie przy zastosowaniu odpowiednich przekładni mechanicznych.

Na podstawie analizy dostępnej literatury oraz porównania właściwości silników krokowych z innymi typami napędów, możliwe jest wskazanie kluczowych zalet i ograniczeń tego rozwiązania w kontekście budowy manipulatora [10]. Poniżej zestawiono najważniejsze cechy, które zostały uwzględnione podczas rozważań:

Zalety zastosowania silników krokowych:

- **Duży moment obciążający:** Przy zastosowaniu odpowiedniej przekładni uzyskuje się znacznie większy moment niż w najwyższej klasy serwomechanizmach.
- **Wysoka dokładność:** Silniki krokowe oferują rozdzielczość około $1,8^\circ$ na krok, co w połączeniu z przekładnią umożliwia osiągnięcie bardzo dużej dokładności ruchów.

- **Redukcja luzów:** Przekładnie pasowe generują luz praktycznie pomijalny, zaś przekładnie sferyczne i cykloidalne, w zależności od jakości wykonania, również charakteryzują się bardzo małym luzem.

Wady zastosowania silników krokowych:

- **Koszt:** Silniki krokowe wraz z przekładniami i sterownikami stanowią znaczące wydatki. Koszt kompletu silnika i przekładni może wielokrotnie przewyższać cenę serwomechanizmu o porównywalnym momencie obciążającym. Dodatkowo silniki wymagają dedykowanych sterowników.
- **Masa:** Silniki krokowe, nawet bez przekładni, ważą 5–6 razy więcej niż wysokojakościowe serwomechanizmy.
- **Gabaryty:** Większe rozmiary silników krokowych zwiększają gabaryt całej konstrukcji przegubów, a także ryzyko kolizji wewnątrz manipulatora.
- **Złożoność sterowania:** Silnik krokowy wymaga informacji o bieżącej pozycji, ponieważ sterowanie ogranicza się do polecenia wykonania określonej liczby kroków lub półkroków. Konieczne jest zatem wykonanie sekwencji inicjalizacyjnej polegającej na najeźdźaniu do pozycji krańcowej (np. poprzez styk lub czujnik), aby system znalazł punkt referencyjny.
- **Utrata kroków:** Silniki krokowe mogą gubić kroki, co wymaga implementacji pętli sprzężenia zwrotnego (np. za pomocą enkoderów zamontowanych na przegubach), dodatkowo komplikując konstrukcję.

2.1.2. Rozwiązanie nr 2 - serwomechanizmy

Serwomechanizmy są jednym z najczęściej stosowanych rozwiązań napędowych w lekkich manipulatorach robotycznych, szczególnie w konstrukcjach prototypowych, edukacyjnych oraz hobbystycznych. Ich popularność wynika z prostej integracji z systemami mikrokontrolerowymi oraz dużej dostępności rynkowej. Serwomechanizmy oferowane są w szerokim zakresie wariantów, obejmujących różne wymiary obudów, zakresy kątowe pracy (zwykle od 90° do 270°), momenty obrotowe, a także wersje z przekładniami plastikowymi lub metalowymi, co umożliwia ich dobór do zróżnicowanych zastosowań.

Dodatkowym atutem serwomechanizmów jest wbudowana elektronika sterująca, realizująca podstawową pętlę regulacji położenia, dzięki czemu sterowanie sprowadza się do zadawania pozycji kątowej za pomocą sygnału PWM. Rozwiązanie to znacząco upraszcza architekturę systemu sterowania [11]. Na podstawie analizy dostępnych rozwiązań możliwe jest wyróżnienie następujących zalet i ograniczeń serwomechanizmów.

Zalety zastosowania serwomechanizmów:

- **Niska cena:** Serwomechanizmy są ekonomicznie opłacalne.
- **Niska masa:** Charakteryzują się minimalnym ciężarem.
- **Kompaktowe wymiary:** Niewielkie rozmiary ułatwiają integrację w ograniczonej przestrzeni.

- **Łatwe sterowanie:** Nie wymagają pętli sprzężenia zwrotnego, pamiętają ostatnią pozycję i nie potrzebują sekwencji inicjalizacyjnej.

Wady zastosowania serwomechanizmów:

- **Ograniczony moment obciążający:** Serwomechanizmy nie generują dużych momentów.
- **Znaczna luz:** Transmisje serwomechanizmów charakteryzują się dużym luzem. W połączeniu z rozdzielczością sterowania (około 2°) oraz dodatkowym luzem na przekładni (empirycznie oszacowanym na około 2°), całkowita luz osiąga wartości uniemożliwiające zastosowanie w profesjonalnych rozwiązaniach.
- **Zmienność jakości:** Każdy egzemplarz serwomechanizmu różni się parametrami – luz, osadzenie wielowypustu na wale wyjściowym (utrudniające precyzyjne wycentrowanie adapterów montażowych), zakresy pracy często znacząco odbiegające od deklaracji producenta.

2.1.3. Rozwiązanie nr 3 - hybrydowe

Rozwiązanie hybrydowe zakładało zastosowanie silników krokowych w przegubach 0 i 1, czyli w miejscach przenoszących największe obciążenia oraz mających kluczowy wpływ na dokładność pozycjonowania efektora końcowego. Przeguby te odpowiadają za pozycjonowanie całego ramienia manipulatora w przestrzeni, a ewentualne błędy lub luzy w ich pracy powodują istotne odchylenia położenia efektora. Zastosowanie silników krokowych w tych przegubach umożliwiłoby uzyskanie wysokiej rozdzielczości ruchu oraz dużego momentu obrotowego, szczególnie przy wykorzystaniu przekładni mechanicznych.

Jednocześnie w przegubach 2-5, które przenoszą mniejsze obciążenia i w mniejszym stopniu wpływają na globalną dokładność pozycjonowania efektora, przewidziano zastosowanie serwomechanizmów. Pozwalałoby to na znaczną redukcję masy części ruchomych manipulatora, co przekłada się na mniejsze obciążenie przegubów bazowych, prostszą konstrukcję mechaniczną oraz mniejsze wymagania względem układu sterowania.

Takie połączenie obu typów napędów umożliwiłoby wykorzystanie zalet silników krokowych w newralgicznych przegubach przy jednoczesnym ograniczeniu ich wad, takich jak duża masa, gabaryty oraz złożoność sterowania. Z kolei zastosowanie serwomechanizmów w dalszych przegubach pozwalałoby zachować prostotę sterowania i niską masę, minimalizując wpływ ich ograniczeń na dokładność całego układu. W efekcie rozwiązanie hybrydowe stanowiłoby kompromis pomiędzy precyzją, masą konstrukcji oraz złożonością systemu sterowania.

2.1.4. Ostateczny wybór

Po analizie przedstawionych wariantów napędu oraz uwzględnieniu założeń projektowych zdecydowano się na rozwiązanie oparte wyłącznie na zastosowaniu serwomechanizmów. Decyzja ta była podyktowana przede wszystkim ograniczeniami czasowymi, finansowymi oraz chęcią do ograniczenia złożoności konstrukcji mechanicznej i systemu sterowania.

Zastosowanie serwomechanizmów umożliwiło znaczące uproszczenie architektury układu sterowania, eliminując konieczność stosowania dodatkowych sterowników silników krokowych, enkoderów oraz procedur inicjalizacji pozycji referencyjnej. Pozwoliło to na skupienie się na integracji manipulatora ze środowiskiem ROS 2 oraz implementacji funkcjonalności planowania i realizacji ruchu, które stanowią główny zakres niniejszej pracy.

Przegub 0 napędzany jest serwomechanizmem o nieograniczonym zakresie obrotu (360°), będącym w praktyce silnikiem prądu stałego z przekładnią oraz wbudowanym układem sterującym. Transmisja momentu została zrealizowana za pomocą przekładni zębatej o przełożeniu 1:3, co pozwoliło na zwiększenie dostępnego momentu obrotowego oraz poprawę rozdzielczości sterowania. Sprzężenie zwrotne położenia zaimplementowano przy użyciu potencjometru zamontowanego współosiowo z osią przegubu, co umożliwia realizację podstawowej pętli regulacji położenia.

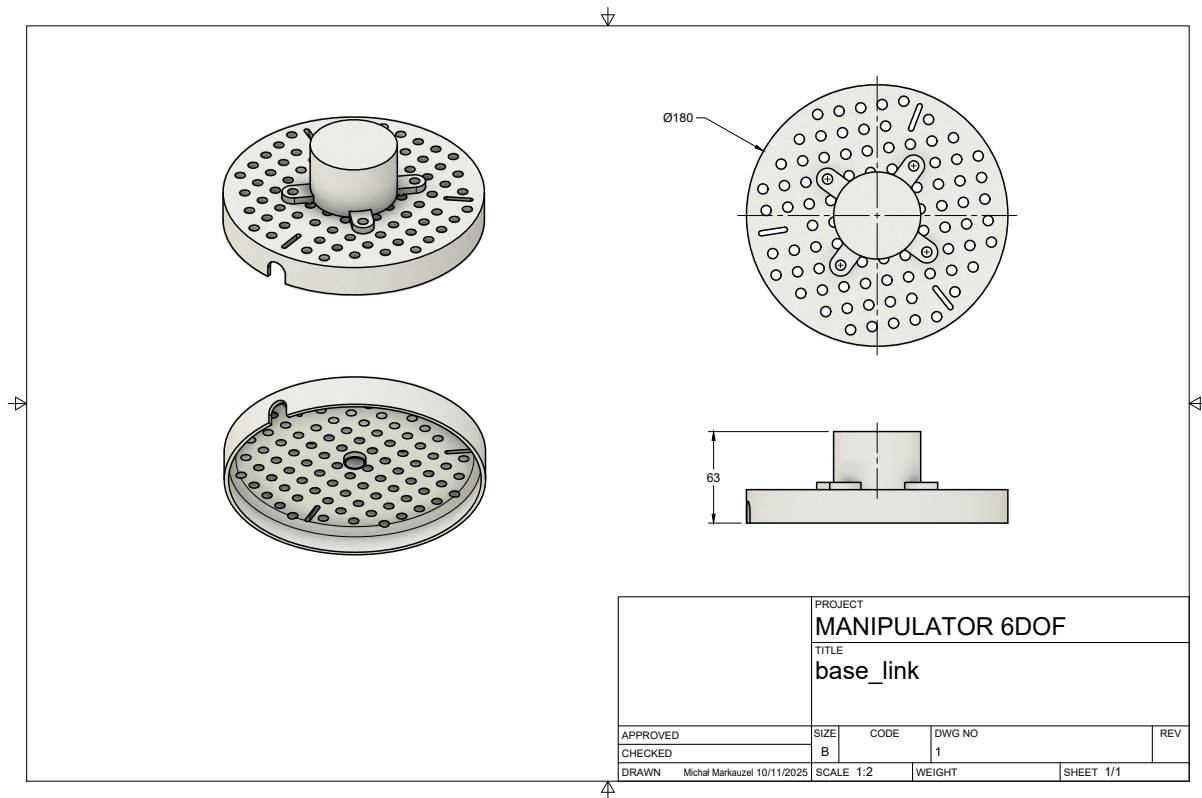
Pozostałe przeguby manipulatora (1-5) sterowane są standardowymi serwomechanizmami bez dodatkowego sprzężenia zwrotnego. Takie rozwiązanie pozwoliło na redukcję masy członów ruchomych, uproszczenie konstrukcji mechanicznej oraz ograniczenie liczby elementów elektronicznych, przy jednoczesnym zachowaniu funkcjonalności wymaganej do realizacji celów pracy.

2.2. Opis budowy

2.2.1. Człon nr 0 - podstawa

Podstawę stanowi żeliwny obciążnik, który zapewnia stabilność i zwiększoną masę bazowego członu. Na podstawie zamontowana została wydrukowana obudowa zawierająca wiele otworów przygotowanych do wtopienia mosiężnych wkładek gwintowanych (instertów), umożliwiające przyszłe rozszerzenia i dodatki montażowe. Do obudowy zamocowany jest serwomechanizm 360° napędzający przegub 0 oraz ogranicznik zakresu ruchów ochroniacy potencjometr zamontowany od strony dolnej przegubu.

Na rysunku 2.1 przedstawiono rysunek poglądowy członu bazowego.

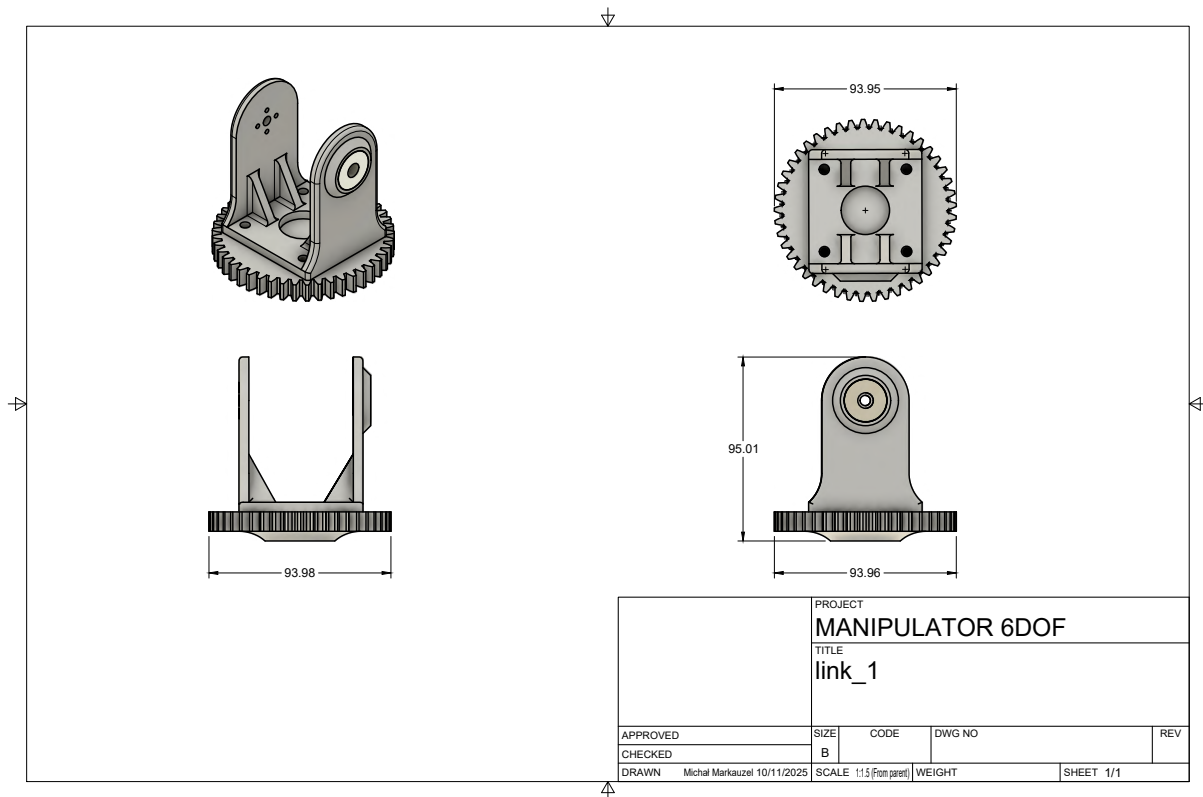


Rysunek 2.1: Rysunek poglądowy członu bazowego (człon 0)

2.2.2. Człon nr 1

Człon nr 1 przymocowany jest do podstawy za pomocą współosiowego układu dwóch łożysk o dużej średnicy, które zapewniają wysoką sztywność konstrukcji oraz precyzyjne prowadzenie osi obrotu. Regulacja luzu osiowego realizowana jest poprzez osiowy docisk elementów konstrukcyjnych od góry i dołu, co umożliwia eliminację luzów montażowych oraz stabilną pracę przegubu. Przegub 1 napędzany jest serwomechanizmem (MG996R, z zakresem 360° , moment 15 kg/cm). Moment obrotowy przenoszony jest za pośrednictwem przekładni o przełożeniu 1:3, co pozwala na zwiększenie momentu kosztem prędkości obrotowej oraz poprawę kontroli ruchu członu.

Na rysunku 2.2 przedstawiono konstrukcję członu nr 1.

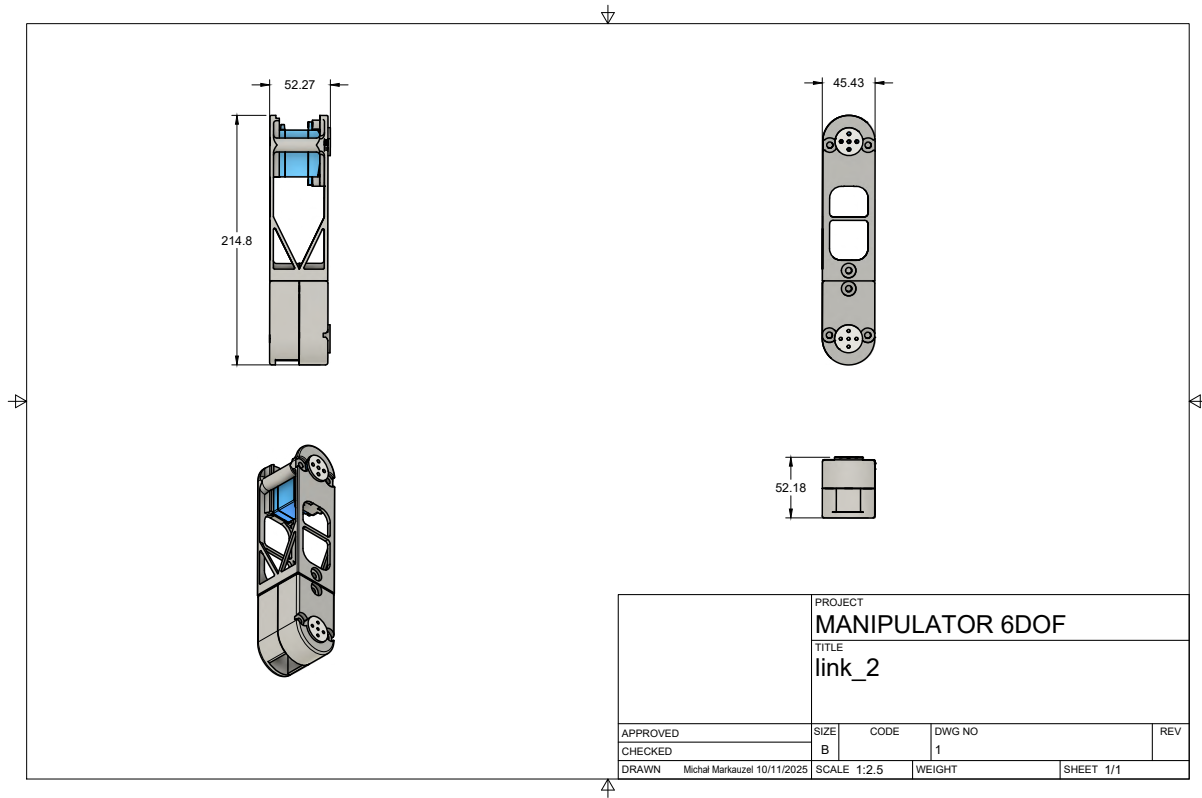


Rysunek 2.2: Człon nr 2 manipulatora

2.2.3. Człon nr 2

Człon nr 2 zbudowany jest z czterech elementów połączonych za pomocą śrub i wkładek mosiężnych. Stanowią go dwa serwomechanizmy (DEAO, z zakresem ruchu 270° , moment 30 kg/cm). Przegub łączy poprzedni i kolejny człon poprzez aluminiowy adapter serwomechanizmu oraz łożysko 608ZZ, zapewniające sztywność i osiowość złącza. Konstrukcja została zredukowana pod względem masy w celu zmniejszenia obciążenia przegubów.

Na rysunku 2.3 przedstawiono konstrukcję członu nr 2.

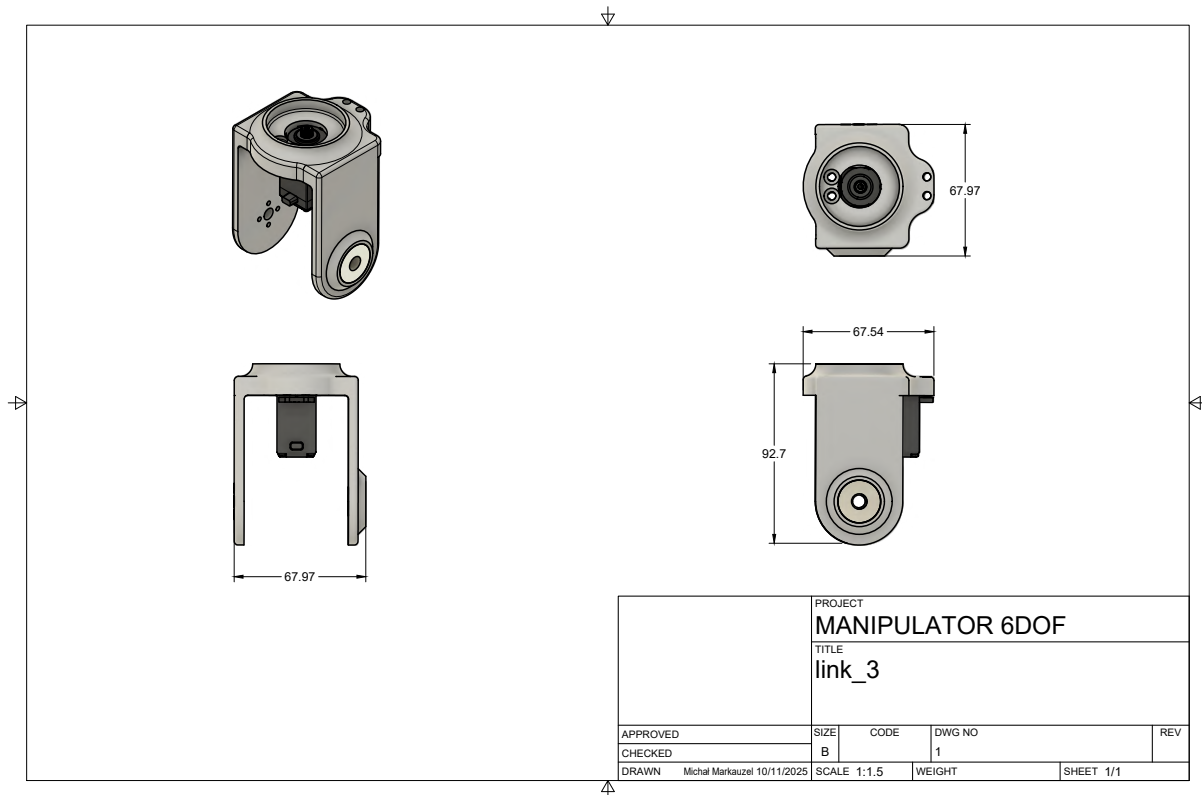


Rysunek 2.3: Człon nr 2 manipulatora

2.2.4. Człon nr 3

Człon nr 3 zaprojektowano analogicznie do członu nr 1. Jest on zamocowany do serwomechanizmu znajdującego się w poprzednim członie, natomiast z przeciwnej strony osadzony na łożysku 608ZZ w celu zapewnienia osiowości pracy przegubu. W konstrukcji członu nr 3 przewidziano również mocowanie serwomechanizmu serwomechanizmem (MG996R, z zakresem 180° , moment 15 kg/cm) odpowiedzialnego za obrót członu nr 4 oraz gniazdo łożyska umożliwiające zamocowanie kolejnego członu manipulatora.

Na rysunku 2.4 przedstawiono konstrukcję członu nr 3.

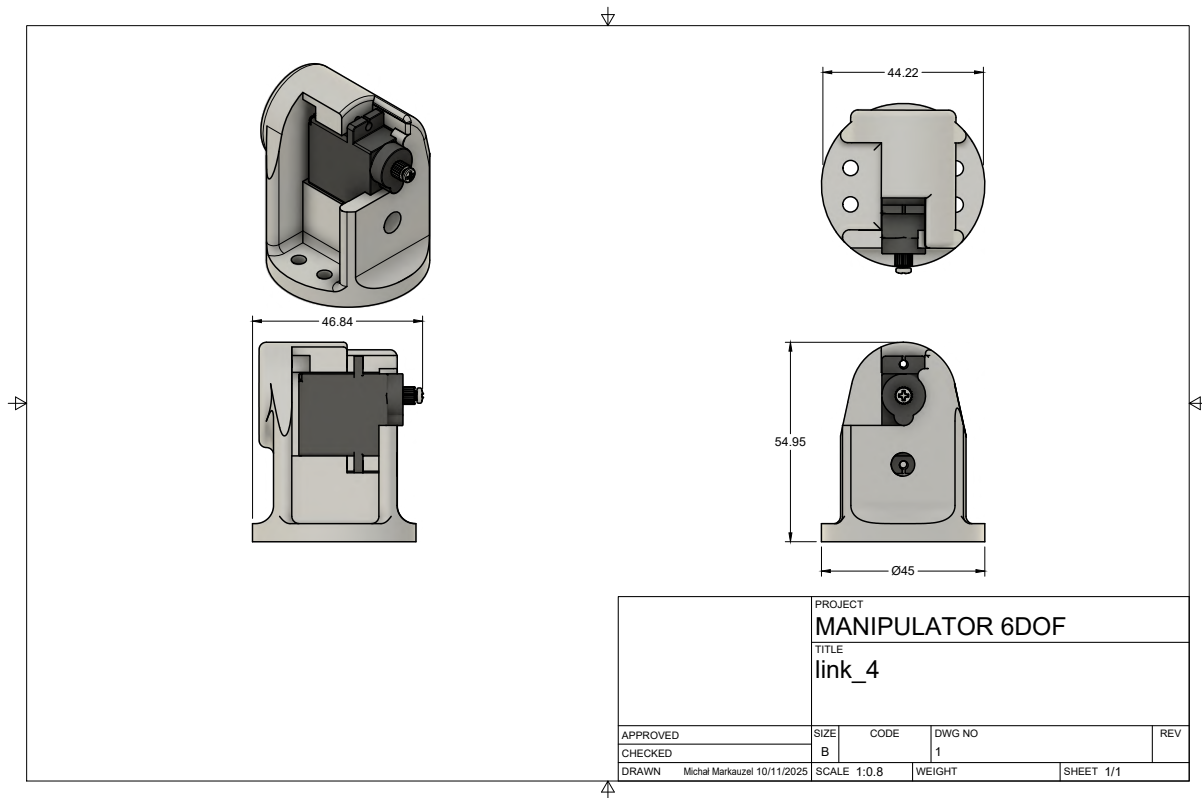


Rysunek 2.4: Człon nr 3 manipulatora

2.2.5. Człon nr 4

W członie nr 4 zastosowano lżejszy serwomechanizm (MG90S, z zakresem 180° , moment $2,2 \text{ kg/cm}$) w celu redukcji masy spoczywającej nad przegubami poprzedzającymi. Serwomechanizm osadzony jest na jednym łożysku zamontowanym w członie nr 3 metodą wciskania (press-fit), co powoduje nieznaczne zmniejszenie sztywności i osiowości przegubu w porównaniu z przegubami poprzedzającymi.

Na rysunku 2.5 przedstawiono konstrukcję członu nr 4.

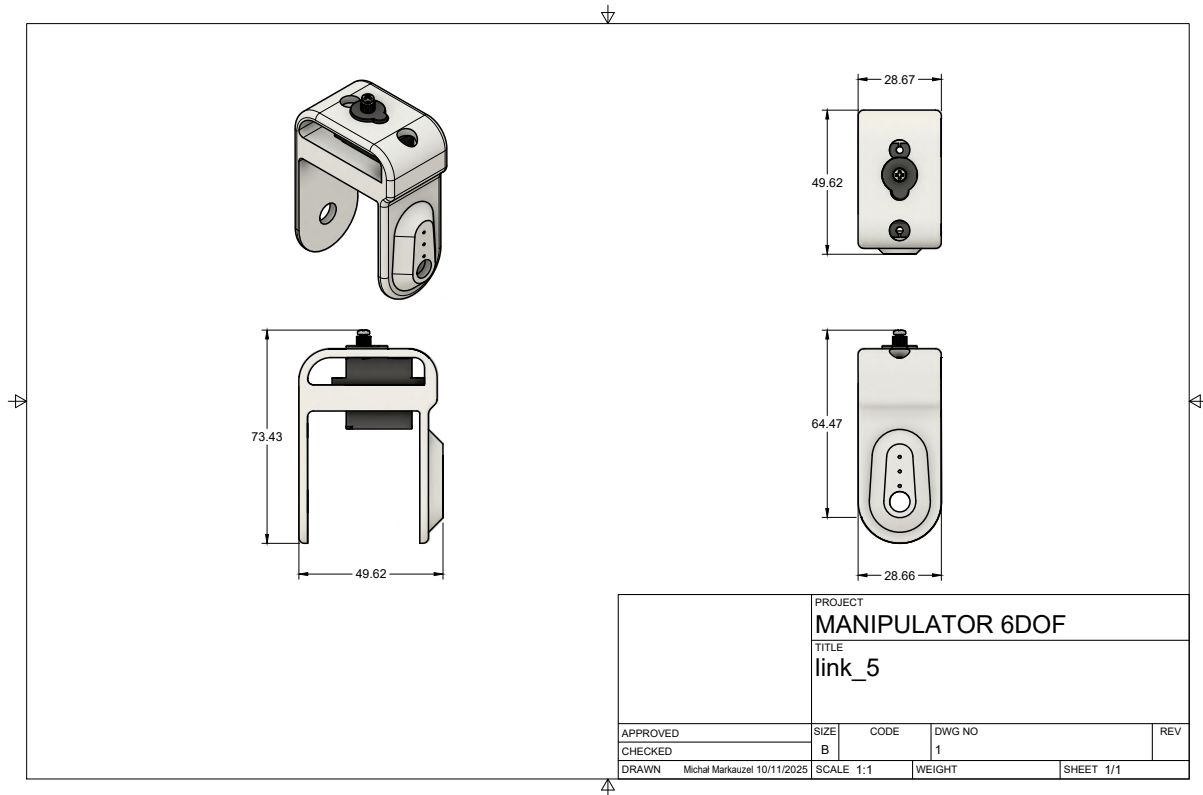


Rysunek 2.5: Człon nr 4 manipulatora

2.2.6. Człon nr 5

Człon nr 5 wyposażony jest również w lżejszy serwomechanizm (MG90S, z zakresem 180° , moment $2,2 \text{ kg/cm}$). Przymocowany jest do członu nr 4 w dwóch punktach: poprzez wielowypust na wale serwomechanizmu oraz za pośrednictwem łożyska 608ZZ.

Na rysunku 2.6 przedstawiono konstrukcję członu nr 5.

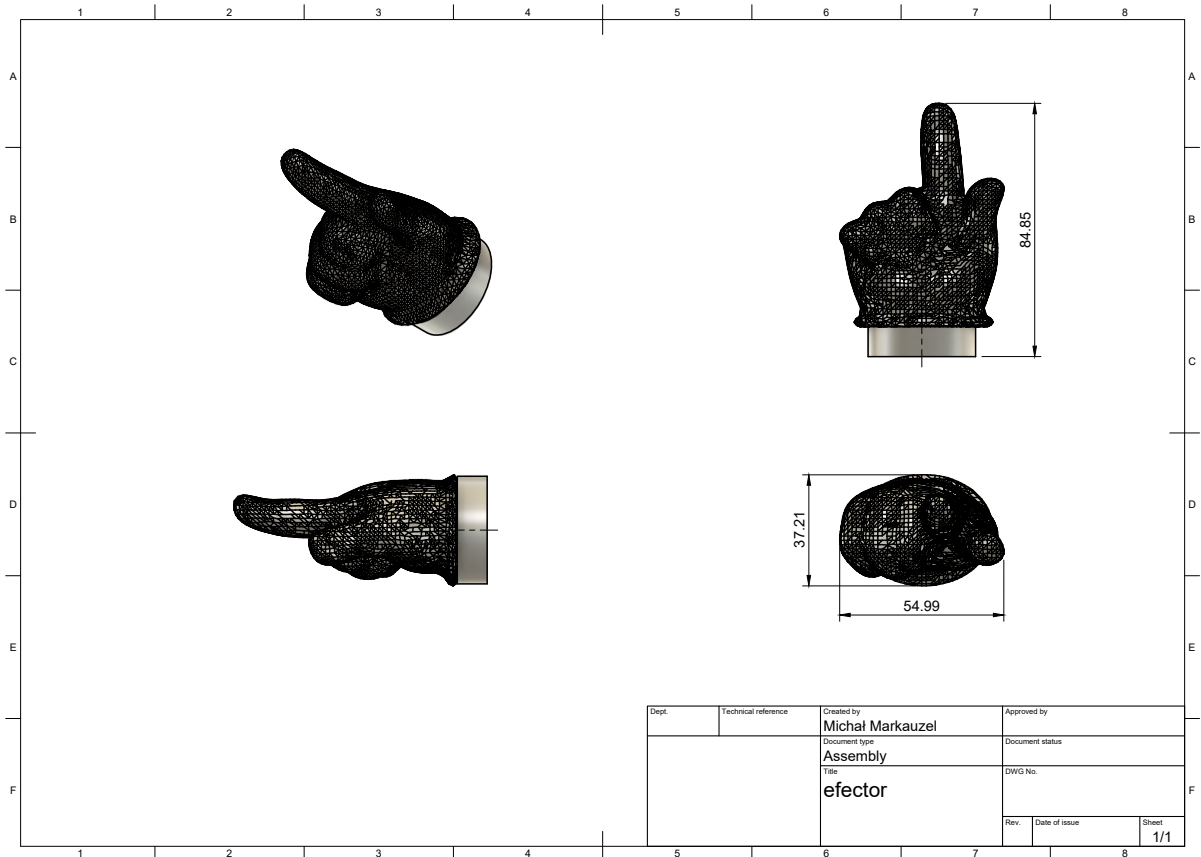


Rysunek 2.6: Człon nr 5 manipulatora

2.2.7. Człon nr 6 – efektor

Człon nr 6 przymocowany jest bezpośrednio do ostatniego serwomechanizmu, gdyż nie przenosi znaczących obciążeń i nie wymaga wysokiej sztywności i osiowości. Rozwiązanie to pozwala na zmniejszenie masy w miejscu działania największej dźwigni, co redukuje obciążenie przegubu 1. Do członu nr 6 przymocowany jest model dłoni poprzez wkręcenie do wtopionej mosiężnej wkładki M5.

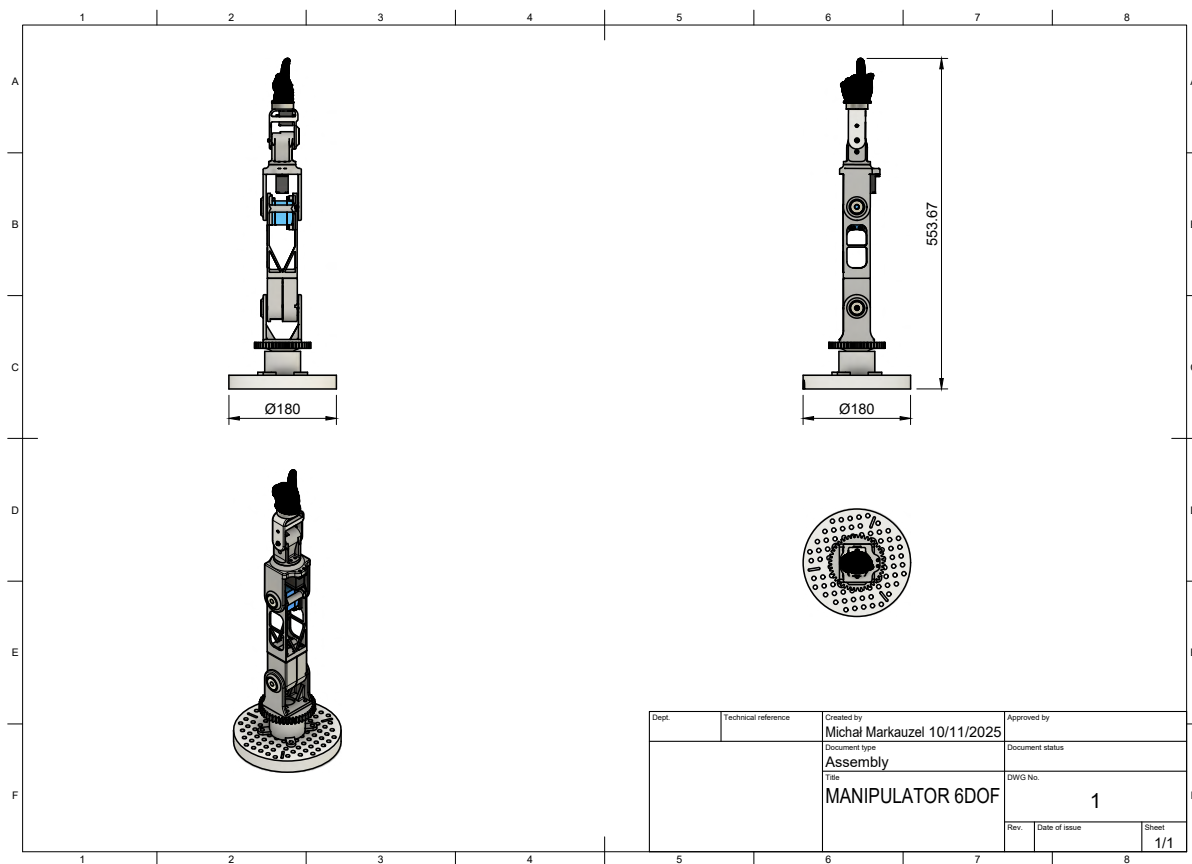
Na rysunku 2.7 przedstawiono konstrukcję efektora końcowego.



Rysunek 2.7: Efektor końcowy manipulatora (człon 6)

2.3. Złożenie całkowite

Na rysunku 2.8 przedstawiono złożenie manipulatora ze wszystkimi podzespołami.



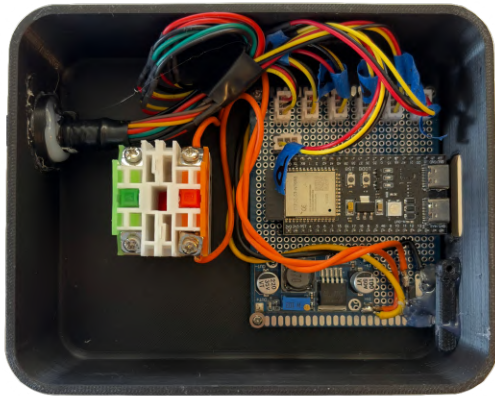
Rysunek 2.8: Kompletnie złożenie manipulatora

2.4. Układ sterujący

Układ sterujący został zrealizowany w oparciu o mikrokontroler ESP32-S3 N16R8, umieszczony w dedykowanej obudowie. Z punktu widzenia sprzętowego system udostępnia sześć linii sygnałowych PWM, przeznaczonych do sterowania serwomechanizmami przegubów 0–5. Dodatkowo do mikrokontrolera doprowadzony jest jeden kanał analogowy, wykorzystywany do odczytu sygnału z potencjometru pełniącego funkcję sprzężenia zwrotnego położenia przegubu bazowego.

Układ wyposażono w przycisk awaryjny, umożliwiający natychmiastowe odcięcie napięcia zasilania części wykonawczej manipulatora. Zasilanie realizowane jest poprzez gniazdo żeńskie, do którego podłączany jest zewnętrzny zasilacz (6V, 6A). Programowanie mikrokontrolera oraz komunikacja z systemem nadrzędnym realizowane są za pomocą dwóch złączy USB-C, zapewniających interfejs programowania oraz transmisję danych w trybie szeregowym.

Połączenie układu sterującego z manipulatorem zrealizowano za pomocą wiązki przewodów poprowadzonej do podstawy robota, umożliwiającej dystrybucję sygnałów sterujących oraz zasilania do poszczególnych przegubów. Rysunki 2.9 i 2.10 przedstawiają układ sterujący odpowiednio od strony wewnętrznej i zewnętrznej.



Rysunek 2.9: Układ sterowania – widok od strony wewnętrznej



Rysunek 2.10: Układ sterowania – widok od strony zewnętrznej

2.5. Rzeczywisty model manipulatora

Na rysunku 2.15 przedstawiono rzeczywisty model manipulatora w kilku konfiguracjach roboczych oraz z różnych perspektyw.



Rysunek 2.11: Pozycja nr 1



Rysunek 2.12: Pozycja pos_zero



Rysunek 2.13: Pozycja nr 2



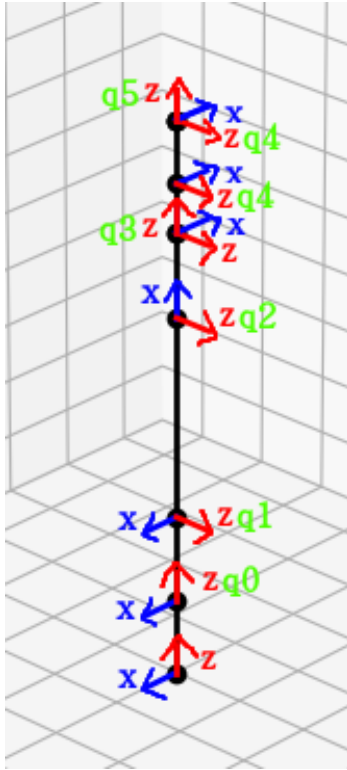
Rysunek 2.14: Pozycja nr 3

Rysunek 2.15: Rzeczywisty model fizyczny manipulatora w różnych konfiguracjach roboczych

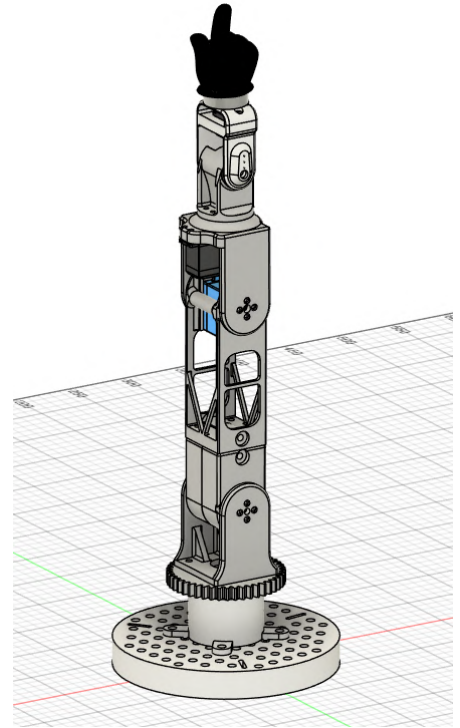
3. Analiza kinematyki manipulatora

3.1. Schemat Kinematyczny

Analiza kinematyczna manipulatora została przeprowadzona w oparciu o notację Denavita-Hartenberga [12, 13, 14, 15, 16]. Na rysunku 3.1 przedstawiono schemat kinematyczny manipulatora w pozycji zerowej, który stanowi podstawę dalszych rozważań analitycznych. Dla porównania, na rysunku 3.2 zaprezentowano model manipulatora w tej samej konfiguracji.



Rysunek 3.1: Schemat kinematyczny manipulatora w pozycji zerowej



Rysunek 3.2: Model manipulatora w pozycji zerowej

3.2. Tabela parametrów Denavita–Hartenberga

Na podstawie modelu 3D manipulatora wyznaczono tabelę parametrów Denavita–Hartenberga w klasycznej notacji DH. Wszystkie przeguby są obrotowe, dlatego zmienne konfiguracyjne opisane są wektorem

$$q = [q_0, q_1, q_2, q_3, q_4, q_5]^T.$$

Zastosowanie dodatkowych układów pośrednich skutkuje istnieniem ramek $\{0\} \dots \{9\}$, przy czym tylko wybrane parametry θ_i zależą od zmiennych konfiguracyjnych.

Tabela 3.1: Parametry Denavita–Hartenberga manipulatora

Transformacja	θ_i [rad]	d_i [mm]	a_i [mm]	α_i [rad]
$0 \rightarrow 1$	0	d_1	0	0
$1 \rightarrow 2$	q_0	d_2	0	$-\frac{\pi}{2}$
$2 \rightarrow 3$	$q_1 - \frac{\pi}{2}$	0	a_3	0
$3 \rightarrow 4$	q_2	0	a_4	0
$4 \rightarrow 5$	$-\frac{\pi}{2}$	0	0	$-\frac{\pi}{2}$
$5 \rightarrow 6$	q_3	d_5	0	$\frac{\pi}{2}$
$6 \rightarrow 7$	$q_4 + \frac{\pi}{2}$	0	a_6	0
$7 \rightarrow 8$	$-\frac{\pi}{2}$	0	0	$-\frac{\pi}{2}$
$8 \rightarrow 9$	q_5	0	0	0

Wartości parametrów geometrycznych występujących w tabeli, tj. d_i oraz a_i , przyjmują następujące wartości:

$$\begin{aligned} d_1 &= 63.0 \text{ mm}, & d_2 &= 72.5 \text{ mm}, & d_5 &= 40.5 \text{ mm}, \\ a_3 &= 169.4 \text{ mm}, & a_4 &= 71.0 \text{ mm}, & a_6 &= 52.5 \text{ mm}. \end{aligned}$$

3.3. Macierze Transformacji

3.3.1. Ogólna macierz transformacji A_{i-1}^i

Zgodnie z konwencją Denavita–Hartenberga, każda macierz transformacji A_{i-1}^i , opisująca położenie i orientację układu współrzędnych i względem układu $i-1$, jest iloczynem czterech podstawowych transformacji geometrycznych:

$$A_{i-1}^i = Rot(Z, \theta_i) Trans(Z, d_i) Trans(X, a_i) Rot(X, \alpha_i).$$

Poniżej znajduje się rozwinięcie tego iloczynu:

- Rotacja wokół osi Z

$$Rot(Z, \theta_i) = \begin{pmatrix} \cos \theta_i & -\sin \theta_i & 0 & 0 \\ \sin \theta_i & \cos \theta_i & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Translacja wzdłuż osi Z

$$Trans(Z, d_i) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Translacja wzdłuż osi X

$$Trans(X, a_i) = \begin{pmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Rotacja wokół osi X

$$Rot(X, \alpha_i) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha_i & -\sin \alpha_i & 0 \\ 0 & \sin \alpha_i & \cos \alpha_i & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Wymnożenie powyższych macierzy w podanej kolejności prowadzi do ogólnej postaci macierzy transformacji Denavita-Hartenberga:

$$A_{i-1}^i = \begin{pmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

3.3.2. Macierze transformacji dla manipulatora

Na podstawie tabeli 3.1 wyznaczono kolejne macierze transformacji.

$$A_0^1 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$A_1^2 = \begin{pmatrix} \cos q_0 & 0 & -\sin q_0 & 0 \\ \sin q_0 & 0 & \cos q_0 & 0 \\ 0 & -1 & 0 & d_2 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$A_2^3 = \begin{pmatrix} \sin q_1 & \cos q_1 & 0 & a_3 \sin q_1 \\ -\cos q_1 & \sin q_1 & 0 & -a_3 \cos q_1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$A_3^4 = \begin{pmatrix} \cos q_2 & -\sin q_2 & 0 & a_4 \cos q_2 \\ \sin q_2 & \cos q_2 & 0 & a_4 \sin q_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$A_4^5 = \begin{pmatrix} 0 & 0 & 1 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$A_5^6 = \begin{pmatrix} \cos q_3 & 0 & \sin q_3 & 0 \\ \sin q_3 & 0 & -\cos q_3 & 0 \\ 0 & 1 & 0 & d_5 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$A_6^7 = \begin{pmatrix} -\sin q_4 & -\cos q_4 & 0 & -a_6 \sin q_4 \\ \cos q_4 & -\sin q_4 & 0 & a_6 \cos q_4 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$A_7^8 = \begin{pmatrix} 0 & 0 & 1 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$A_8^9 = \begin{pmatrix} \cos q_5 & -\sin q_5 & 0 & 0 \\ \sin q_5 & \cos q_5 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

3.4. Transformacja całkowita T_0^9

Macierz transformacji jednorodnej opisująca położenie oraz orientację układu związanego z efektem końcowym względem układu bazowego została wyznaczona jako iloczyn kolejnych macierzy transformacji cząstkowych, zgodnie z notacją Denavita-Hartenberga:

$$T_0^9 = A_0^1 \cdot A_1^2 \cdot A_2^3 \cdot A_3^4 \cdot A_4^5 \cdot A_5^6 \cdot A_6^7 \cdot A_7^8 \cdot A_8^9.$$

W celu skrócenia zapisu oraz poprawy czytelności dalszych rozważań wprowadzono następujące oznaczenia funkcji trygonometrycznych:

$$\begin{aligned} c_i &= \cos(q_i), & s_i &= \sin(q_i), \\ c_{ij} &= \cos(q_i + q_j), & s_{ij} &= \sin(q_i + q_j). \end{aligned}$$

Po wykonaniu mnożenia otrzymano macierz transformacji całkowitej w postaci:

$$T_0^9 = \begin{pmatrix} R_{xx} & R_{xy} & R_{xz} & T_x \\ R_{yx} & R_{yy} & R_{yz} & T_y \\ R_{zx} & R_{zy} & R_{zz} & T_z \\ 0 & 0 & 0 & 1 \end{pmatrix},$$

gdzie elementy macierzy obrotu R wyrażone są zależnościami:

$$R_{xx} = s_0(c_3s_5 + s_3c_4c_5) + c_0(c_{12}(s_3s_5 - c_3c_4c_5) + s_{12}s_4c_5)$$

$$R_{xy} = s_0(c_3c_5 - s_3s_5c_4) + c_0(c_{12}(s_3c_5 + c_3c_4s_5) - s_{12}s_4s_5)$$

$$R_{xz} = -s_0s_3s_4 + c_0(c_{12}c_4 + s_{12}s_4c_3)$$

$$R_{yx} = -c_0(c_3s_5 + s_3c_4c_5) + s_0(c_{12}(s_3s_5 - c_3c_4c_5) + s_{12}s_4c_5)$$

$$R_{yy} = -c_0(c_3c_5 - s_3s_5c_4) + s_0(c_{12}(s_3c_5 + c_3c_4s_5) - s_{12}s_4s_5)$$

$$R_{yz} = c_0s_3s_4 + s_0(c_{12}c_4 + s_{12}s_4c_3)$$

$$R_{zx} = c_{12}(s_4c_5 - s_3s_5) + s_{12}c_3c_4c_5$$

$$R_{zy} = -c_{12}(s_4s_5 + s_3c_5) - s_{12}c_3c_4s_5$$

$$R_{zz} = c_{12}c_4 - s_{12}s_4c_3$$

Wektor położenia efektora końcowego opisany jest równaniami:

$$T_x = c_0(a_3s_1 + (a_4 + d_5)s_{12} + a_6(s_{12}c_4 + c_{12}s_4c_3)) - a_6s_0s_3s_4$$

$$T_y = s_0(a_3s_1 + (a_4 + d_5)s_{12} + a_6(c_{12}c_4 + s_{12}s_4c_3)) + a_6c_0s_3s_4$$

$$T_z = d_1 + d_2 + a_3c_1 + (a_4 + d_5)c_{12} + a_6(c_{12}c_4 - s_{12}s_4c_3)$$

4. Eksport modelu

4.1. Format URDF/XACRO

Do opisu struktury kinematycznej oraz geometrii zaprojektowanego manipulatora zastosowano format URDF (Unified Robot Description Format), który jest standardowym sposobem opisu robotów w środowisku ROS. Format ten umożliwia jednoznaczne określenie członów robota, ich połączeń kinematycznych, geometrii oraz parametrów dynamicznych, co jest niezbędne do wizualizacji, symulacji oraz planowania ruchu.

W pracy wykorzystano rozszerzenie URDF w postaci języka XACRO (XML Macros), które pozwala na modularny i czytelny podział opisu robota. Pliki XACRO są w trakcie uruchamiania systemu automatycznie przetwarzane do postaci klasycznego pliku URDF, rozumianego przez narzędzia ROS 2, takie jak RViz czy MoveIt 2.

Nagłówek pliku i definicja robota

Plik opisu manipulatora rozpoczyna się od nagłówka XML oraz definicji elementu `robot`, który stanowi główny kontener całego modelu:

```
<?xml version="1.0" ?>
<robot name="manipulator"
      xmlns:xacro="http://www.ros.org/wiki/xacro">
```

Atrybut `name` określa nazwę robota wykorzystywaną w systemie ROS, natomiast przestrzeń nazw `xmlns:xacro` umożliwia korzystanie z mechanizmów XACRO, takich jak dołączanie zewnętrznych plików czy stosowanie makr.

Dołączanie plików XACRO

W celu zwiększenia czytelności oraz modularności projektu opis robota został podzielony na kilka plików XACRO, które są dołączane do pliku głównego:

```
<xacro:include filename="$(find manipulator_desc)/urdf/materials.xacro" />
<xacro:include filename="$(find manipulator_desc)/urdf/manipulator.trans" />
<xacro:include filename="$(find manipulator_desc)/urdf/manipulator.gazebo" />
```

Plik `materials.xacro` zawiera definicje materiałów i kolorów wykorzystywanych podczas wizualizacji modelu. Plik `manipulator.trans` przechowuje parametry związane z transformacjami kinematycznymi robota. Plik `manipulator.gazebo` zawiera elementy specyficzne dla środowiska symulacyjnego Gazebo, takie jak ustawienia fizyki lub pluginy.

Definicja członów manipulatora

Każdy sztywny element manipulatora zdefiniowany jest za pomocą znacznika `link`. Przykładem jest definicja członu bazowego `base_link`:

```
<link name="base_link">
  <inertial>
    <origin xyz="0.0 0.0 0.0" rpy="0 0 0"/>
    <mass value="0.0"/>
    <inertia ixx="0.0" iyy="0.0" izz="0.0"/>
  </inertial>
  <visual>
    <geometry>
      <mesh filename="file://$(find manipulator_desc)/meshes/base_link.stl"
        scale="0.001 0.001 0.001"/>
    </geometry>
  </visual>
</link>
```

Sekcja `inertial` określa właściwości dynamiczne członu, takie jak masa oraz bezwładność. Sekcja `visual` odwołuje się do siatki trójwymiarowej w formacie STL, wygenerowanej na etapie projektowania mechanicznego. Skala 0.001 wynika z konwersji jednostek z milimetrów (CAD) na metry, wymagane w systemie ROS 2. Analogicznie zdefiniowano kolejne człony manipulatora, takie jak `link_1_1`, `link_2_1` aż do efektora końcowego `efector_1`, z indywidualnie dobranymi położeniami geometrii oraz parametrami dynamicznymi.

Definicja przegubów

Połączenia pomiędzy kolejnymi członami manipulatora opisane są za pomocą znaczników `joint`. Przykładem jest przegub łączący człon bazowy z pierwszym ramieniem manipulatora:

```
<joint name="joint_base_to_1" type="revolute">
  <origin xyz="0.0 0.0 0.063" rpy="0 0 0"/>
  <parent link="base_link"/>
  <child link="link_1_1"/>
  <axis xyz="0.0 0.0 1.0"/>
  <limit lower="-2.1293" upper="1.7802"
    effort="10.0" velocity="1.5"/>
</joint>
```

Zastosowany typ `revolute` oznacza przegub obrotowy o skończonym zakresie ruchu, określonym przez parametry `lower` oraz `upper`. Oś obrotu przegubu zdefiniowana jest wektorem `axis`, natomiast element `limit` określa dopuszczalny zakres ruchu, maksymalny moment oraz prędkość kątową przegubu.

W analogiczny sposób zdefiniowano pozostałe przeguby manipulatora, tworząc pełny łańcuch kinematyczny zgodny z konstrukcją mechaniczną robota.

4.2. Proces przygotowania modelu

Model trójwymiarowy manipulatora został opracowany w środowisku Autodesk Inventor. Ze względu na dostępność zaawansowanych wtyczek do eksportu, koniecznym było przeniesienie modelu do środowiska Autodesk Fusion 360. Środowisko to umożliwia wykorzystanie zewnętrznych skryptów automatyzujących proces eksportu do formatu URDF/XACRO.

4.3. Konfiguracja przegubów i układów współrzędnych

Kluczowym etapem przygotowania modelu było prawidłowe ustawienie wszystkich przegubów (joints) oraz ich układów współrzędnych. Wymóg ten wynika z następujących przyczyn:

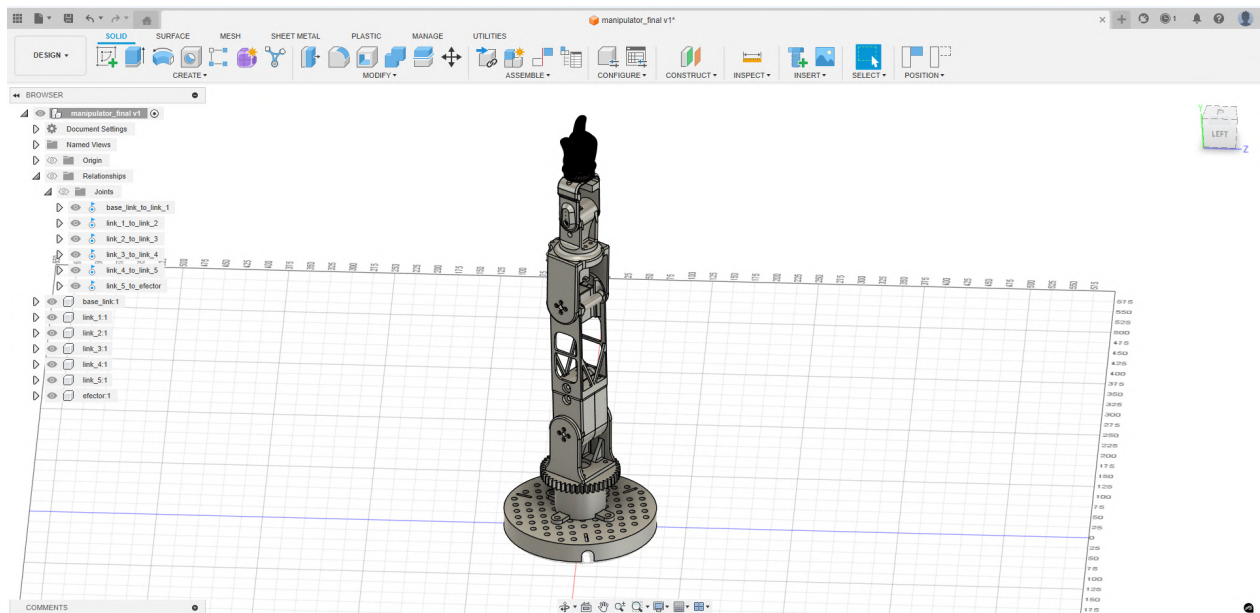
- każdy staw (joint) musi być umieszczony w osi obrotu danego przegłubu,
- początek układu współrzędnych każdego stawu powinien być wyśrodkowany względem ścianek połączenia,
- niedokładne umiejscowienie punktu zerowego (0,0,0) skutkuje błędnym obrotem elementów w środowisku symulacyjnym — model obraca się względem niepoprawnego punktu.

W przypadku stwierdzenia błędów w konfiguracji geometrii lub przegubów konieczne jest ponowne wygenerowanie pliku.

4.4. Eksport przy użyciu skryptu

Po weryfikacji prawidłowego ustawienia wszystkich członów (links) i przegubów (joints) model został wyeksportowany do pakietu ROS 2. Eksport realizowany był przy użyciu skryptu dostępnego w repozytorium GitHub [17].

Na rysunku 4.1 pokazano złożenie manipulatora w środowisku Fusion 360 przed eksportem. Po lewej stronie, w drzewie złożenia, widać wszystkie komponenty: człony (linki) i przeguby (joints). Każdy element został odpowiednio nazwany dla zachowania przejrzystości przy późniejszej pracy z pakietem ROS 2.



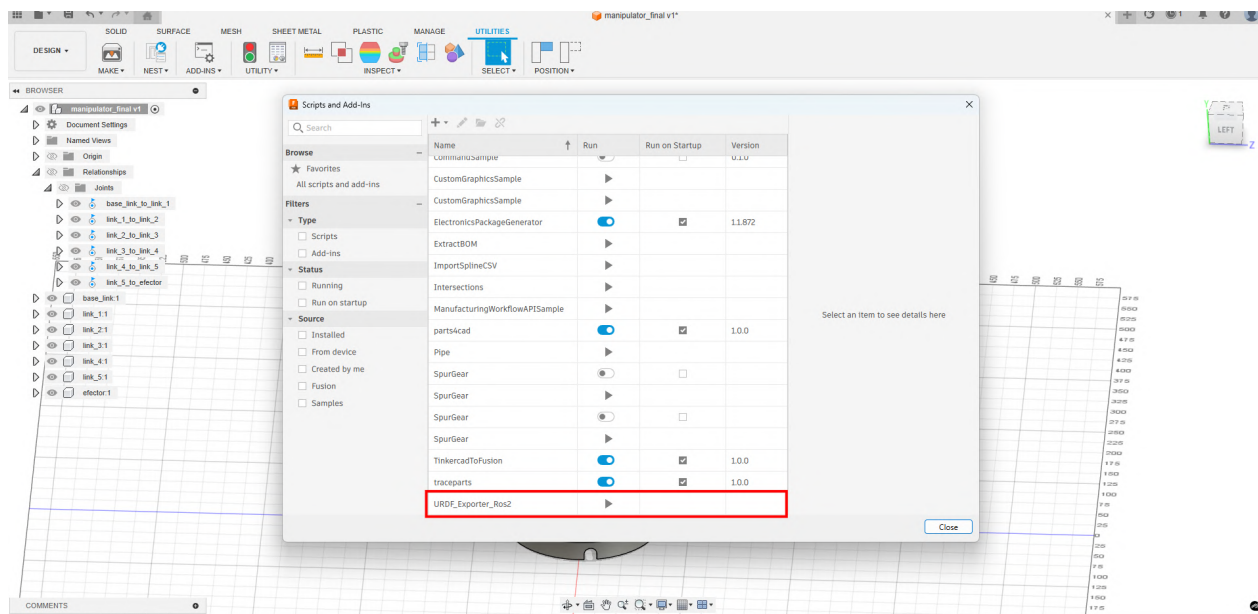
Rysunek 4.1: Podgląd modelu manipulatora w Fusion 360 przed eksportem do formatu URDF

4.5. Uruchomienie procedury eksportu

Eksport modelu realizowano poprzez wykonanie następujących kroków:

1. Przejście do zakładki **ADD-INS** w interfejsie Fusion 360,
2. Wybranie opcji **Scripts**,
3. Uruchomienie skryptu **URDF_ExporterROS 2**.

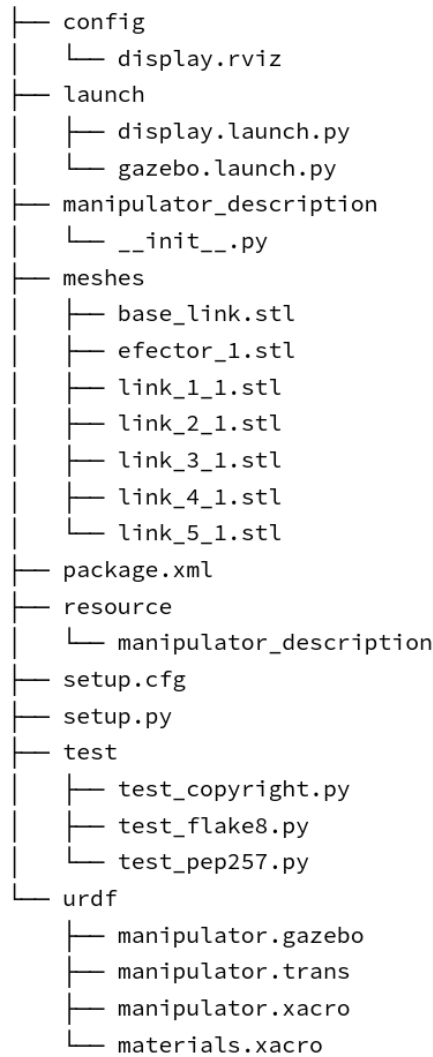
Na rysunku 4.2 przedstawiono interfejs uruchamiania skryptu eksportu. Jeśli model nie zawiera błędów ani niespójności, skrypt automatycznie generuje paczkę ROS 2 w wskazanym katalogu.



Rysunek 4.2: Interfejs uruchamiania skryptu eksportu do formatu URDF

4.6. Struktura wygenerowanego pakietu ROS 2

Rysunek 4.3 przedstawia zawartość struktury katalogów i plików wygenerowanej paczki ROS 2.



Rysunek 4.3: Zawartość wygenerowanej paczki

Paczka zawiera:

- pliki konfiguracyjne (*config*) — ustawienia wizualizacji w RViz,
- pliki uruchamiające (*launch*) — skrypty do włączania symulacji i wizualizacji,
- modele geometryczne (*meshes*) — pliki STL poszczególnych komponentów,
- opis modelu URDF (*urdf*) — definicja struktury kinematycznej manipulatora,
- metadane pakietu — pliki konfiguracyjne projektu ROS 2.

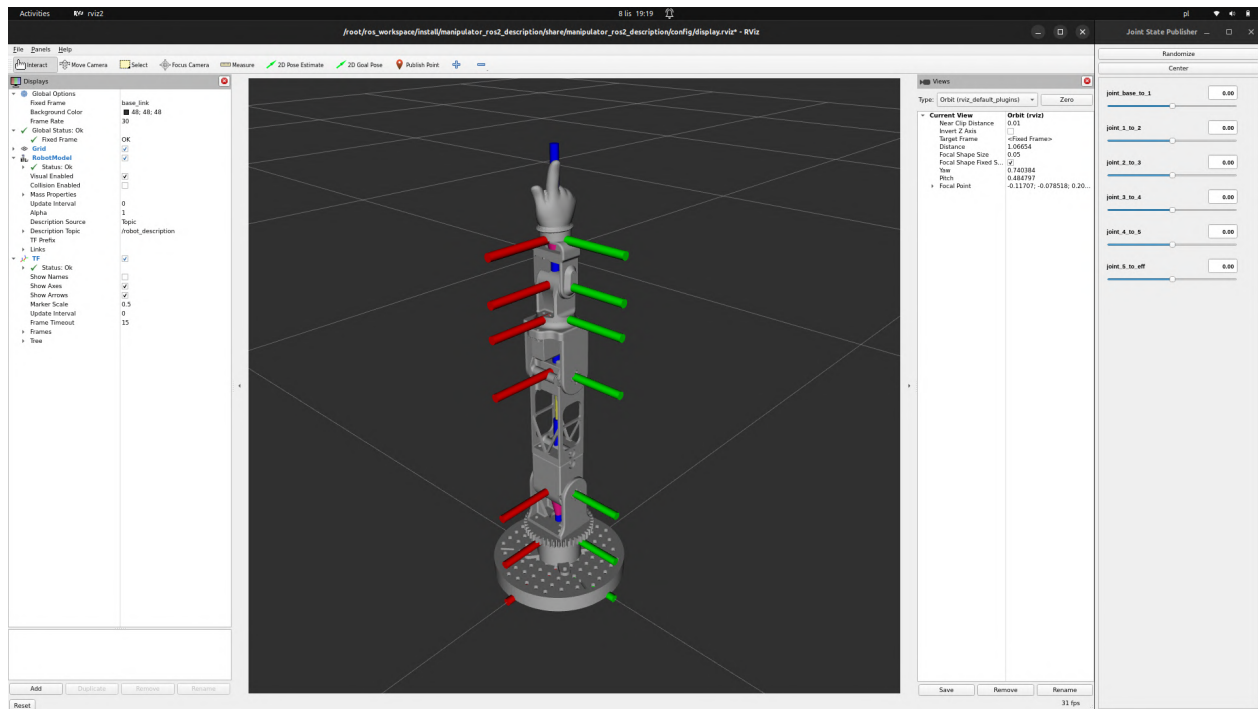
4.7. Weryfikacja modelu w środowisku RViz

Po przeniesieniu pakietu do środowiska ROS 2 możliwe jest uruchomienie wizualizacji za pomocą polecenia:

```
ros2 launch manipulator_ros2_description display.launch.py
```

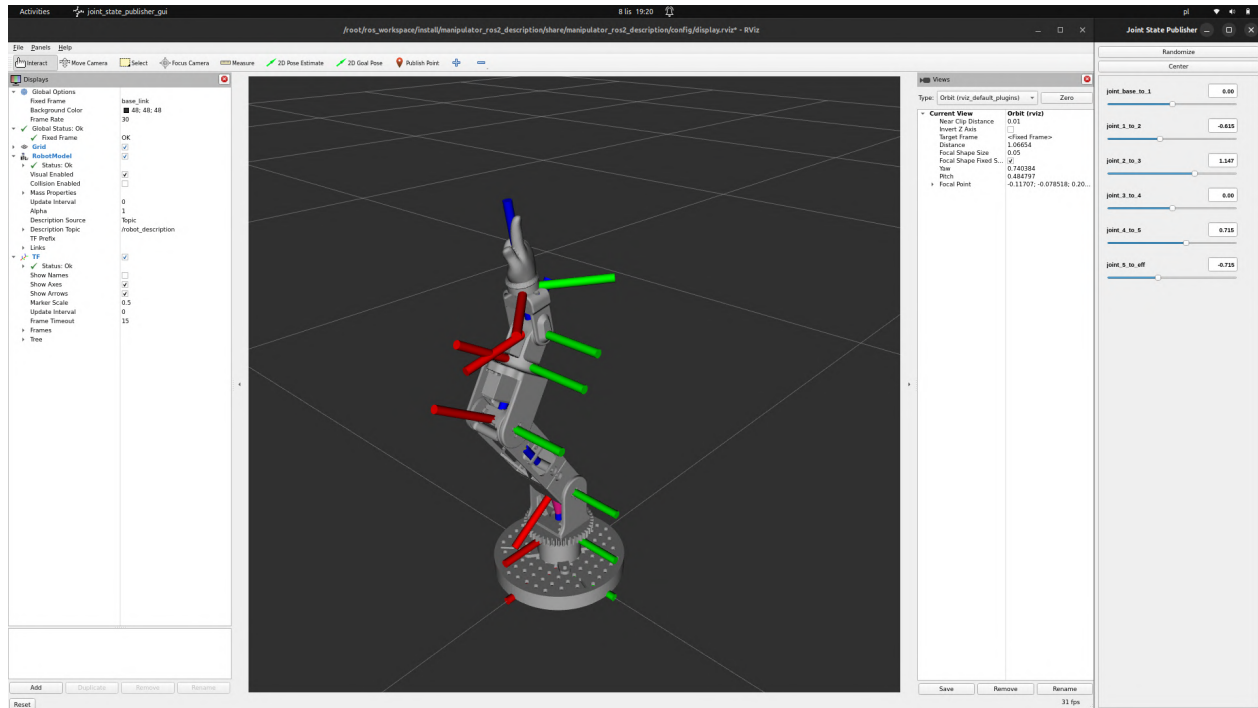
Komenda ta inicjuje środowisko RViz, umożliwiając weryfikację poprawności modelu geometrycznego i kinematycznego.

Na rysunku 4.4 przedstawiono manipulator w środowisku RViz w domyślnym położeniu, z widocznymi osiami współrzędnych każdego członu.



Rysunek 4.4: Manipulator w środowisku RViz w pozycji domyślnej

Na rysunku 4.5 zaprezentowano manipulator po wykonaniu rotacji w każdym z przegłębów. Ilustracja ta potwierdza prawidłową kinematykę i geometrię modelu.



Rysunek 4.5: Manipulator w środowisku RViz po rotacji wszystkich przegłębów

4.8. Podsumowanie

Z przedstawionych ilustracji wynika, że wszystkie osie współrzędnych zostały umieszczone w prawidłowych pozycjach. Po zmianie konfiguracji przegłębów manipulator zachowuje spójność geometryczną i wszystkie komponenty pozostają prawidłowo połączone. Winik ten wskazuje na poprawną implementację modelu URDF oraz możliwość jego wykorzystania w dalszych procesach symulacji w środowisku ROS 2.

5. Tworzenie pakietu MoveIt 2

5.1. Wprowadzenie do platformy MoveIt 2

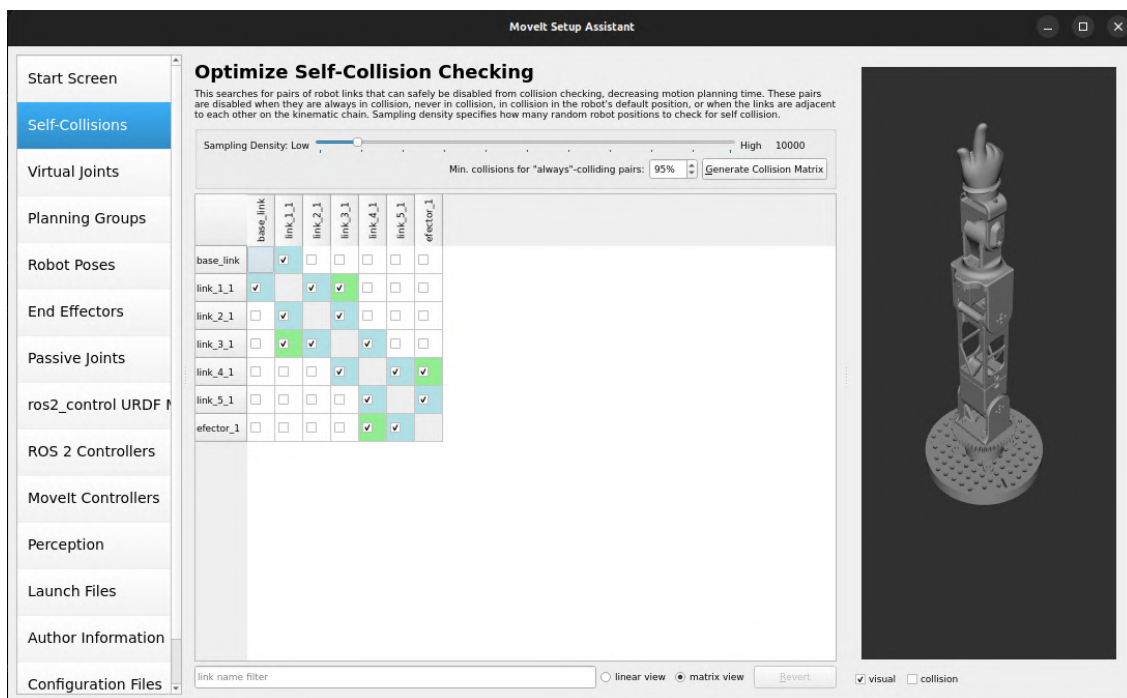
Kolejnym etapem prac było przygotowanie pakietu konfiguracyjnego dla manipulatora przy użyciu platformy MoveIt 2 [18]. Jest to zintegrowana platforma oprogramowania open-source, dedykowana do planowania ruchów (motion planning), manipulacji obiektu, obliczeniach kinematycznych oraz generowaniu trajektorii bezkolizyjnych. Stanowi kluczowy komponent systemu sterowania manipulatorem w środowisku ROS 2. MoveIt 2 udostępnia algorytmy planowania oraz intuicyjne interfejsy programistyczne.

Do wygenerowania niezbędnych plików konfiguracyjnych wykorzystano narzędzie **MoveIt Setup Assistant** (Asystent Konfiguracji MoveIt). Asystent ten jest uruchamiany za pośrednictwem dedykowanego pliku `launch` (`setup_assistant.launch.py`) i prowadzi użytkownika przez serię kroków konfiguracyjnych, bazując na dostarczonym pliku opisu robota w formacie URDF/XACRO. Proces konfiguracji przeprowadzony dla opisywanego manipulatora obejmuje poniższe etapy.

5.2. Optymalizacja detekcji kolizji

Pierwszym krokiem konfiguracji jest optymalizacja sprawdzania kolizji poprzez zdefiniowanie **macierzy autokolizji (Self-Collision Matrix)**. Macierz ta pozwala na wykluczenie z obliczeń par członów (links), które nigdy nie będą ze sobą kolidować z powodu ograniczeń kinematycznych manipulatora lub są ze sobą trwale połączone. Krok ten znacznie redukuje złożoność obliczeniową algorytmów planowania ruchu, skracając czas obliczeń i poprawiając wydajność symulacji.

Na rysunku 5.1 przedstawiono wygenerowaną macierz dla analizowanego manipulatora. Zielone/niebieskie pola oznaczają pary członów permanentnie wyłączone z detekcji kolizji, natomiast białe pola wskazują na pary, które wymagają sprawdzania kolizji podczas planowania.

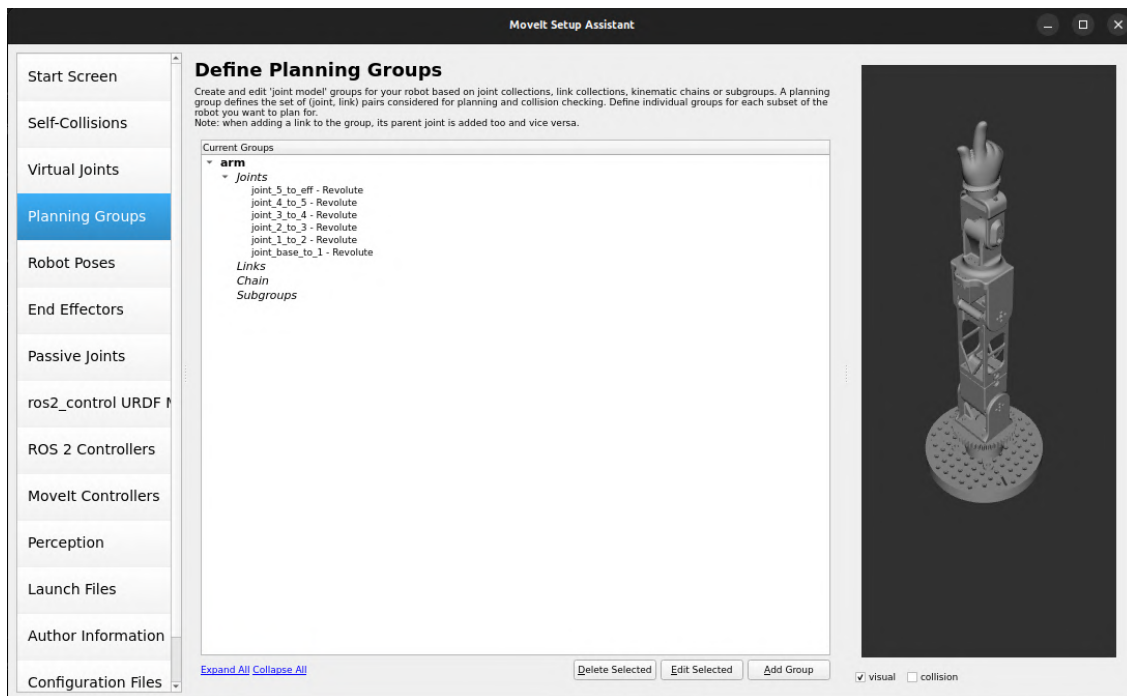


Rysunek 5.1: Interfejs Asystenta MoveIt 2: Wygenerowana macierz autokolizji (Self-Collision Matrix) dla manipulatora

5.3. Definiowanie grup planowania

Kluczowym etapem konfiguracji jest zdefiniowanie **grup planowania (Planning Groups)**. Grupa planowania stanowi spójną jednostkę kinematyczną, będącą zbiorem przegubów (joints) i członów (links), dla której algorytm planowania będzie generować bezkolizyjne trajektorie. Umożliwia to modułowe podejście do sterowania — każda grupa może być planowana i wykonywana niezależnie.

W ramach realizowanego projektu zdefiniowano jedną główną grupę o nazwie **arm**, obejmującą wszystkie sześć przegubów manipulatora (od `joint_base_to_1` do `joint_5_to_eff`). Grupa ta reprezentuje całą kinematyczną strukturę ramienia roboczego. Konfigurację szczegółową przedstawiono na rysunku 5.2.

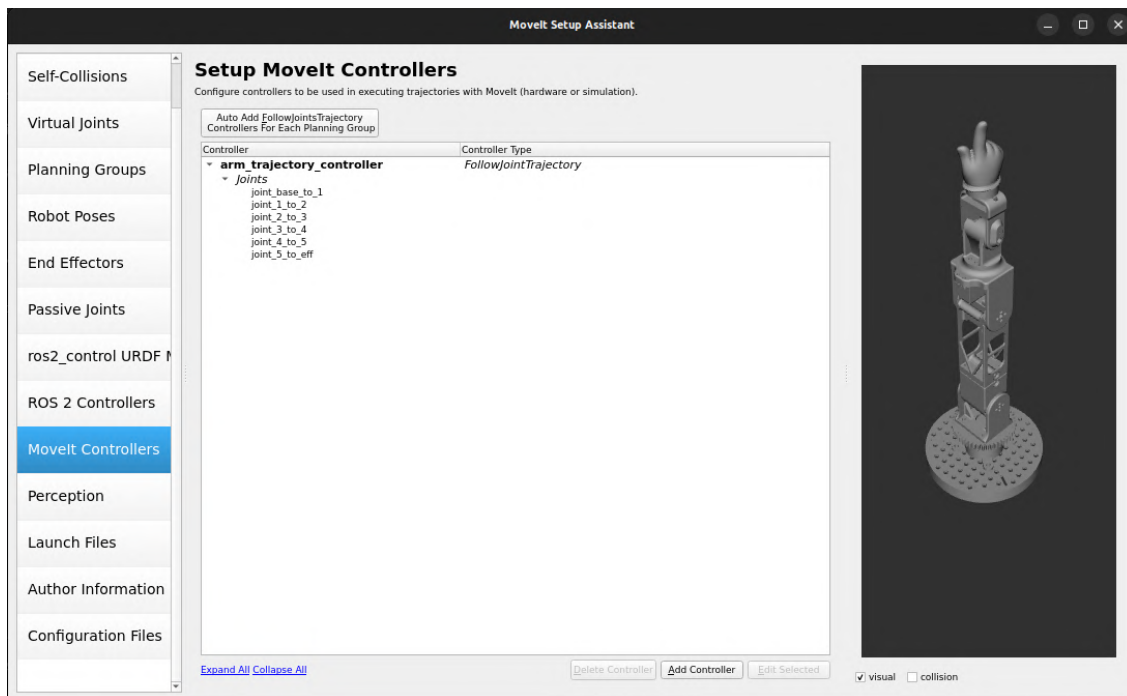


Rysunek 5.2: Interfejs Asystenta MoveIt 2: Definiowanie grup planowania (Planning Groups)

5.4. Konfiguracja kontrolerów

Następnie skonfigurowano kontrolery po stronie MoveIt 2, które są odpowiedzialne za **wykonywanie zaplanowanych trajektorii** w symulacji lub na docelowym sprzęcie. Na tym etapie definiuje się, jaki typ kontrolera steruje daną grupą przegubów oraz jakie parametry są niezbędne do poprawnego wykonania ruchu.

Jak pokazano na rysunku 5.3, dla głównego ramienia manipulatora (kontroler `arm_trajectory_controller`) wybrano standardowy typ `FollowJointTrajectory`. Typ ten implementuje algorytm śledzenia (tracking) zadanej trajektorii przegubów. Konfiguracja generowana automatycznie przez asystent MoveIt 2 często wymaga ręcznej korekcji, aby zapewnić pełną kompatybilność ze sterownikami `ros2_control` i architekturą ROS 2.



Rysunek 5.3: Interfejs Asystenta MoveIt 2: Konfiguracja kontrolerów wykonawczych (MoveIt Controllers)

5.5. Predefiniowane pozy robota

Asystent MoveIt udostępnia możliwość zdefiniowania **predefiniowanych póz robota (Robot Poses)**. Pozy te reprezentują nazwane konfiguracje przegubów, służące jako wygodne punkty odniesienia ułatwiające sterowanie i testowanie manipulatora. Przykładami takich póz są pozycja „domowa”— konfiguracja złożonego robota, czy pozycja „zerowa”— odpowiadająca neutralnemu położeniu wszystkich przegubów.

Na rysunku 5.4 przedstawiono interfejs definiowania póz dla grupy `arm`. Zdefiniowano dwie pozy referencyjne:

- `zero_pos` — odpowiadająca zerowym kątom wszystkich przegubów, wykorzystywana jako punkt zerowy dla testów - pozycja startowa,
- `home_pos` — pozycja domowa umożliwiająca łatwe przechowywanie fizycznego manipulatora.

Predefiniowane pozy znajdują odzwierciedlenie w plikach konfiguracyjnych YAML i mogą być wywoływane programowo lub za pośrednictwem interfejsów użytkownika MoveIt 2.



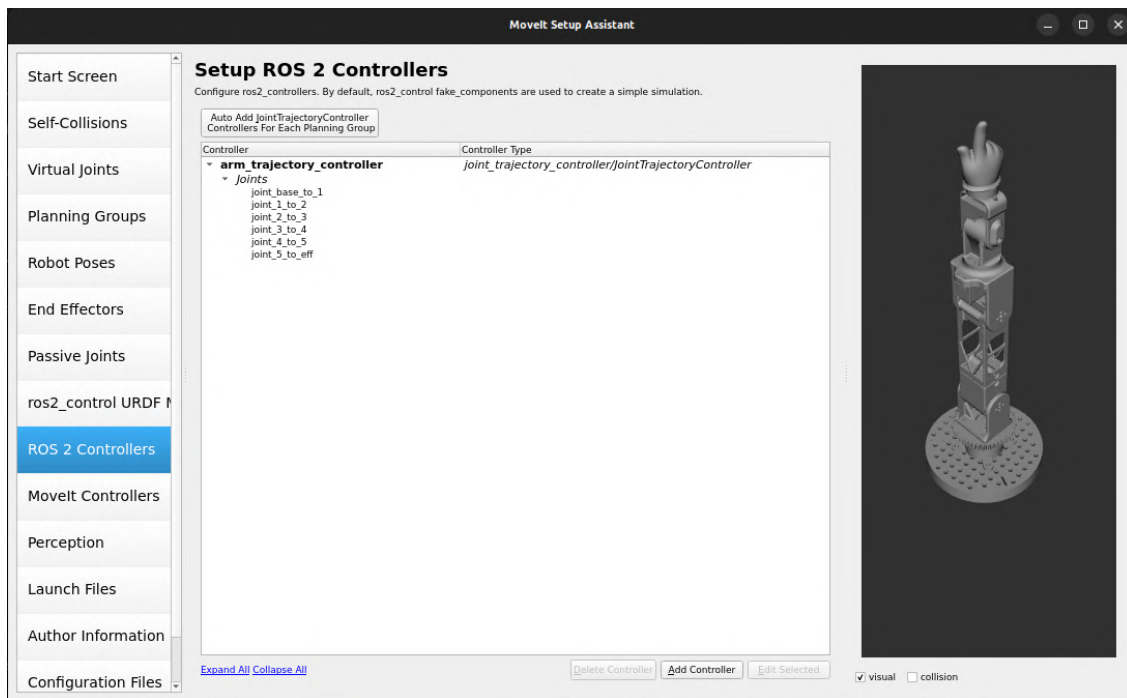
Rysunek 5.4: Interfejs Asystenta MoveIt 2: Definiowanie predefiniowanych póz robota (Robot Poses)

5.6. Konfiguracja kontrolerów ROS 2

Ostatnim istotnym krokiem jest konfiguracja **kontrolerów ROS 2 (ROS 2 Controllers)**, definiująca interfejsy dla frameworka `ros2_control`. Warstwa ta stanowi most pomiędzy algorytmami planowania MoveIt 2 a fizycznym sterowaniem manipulatorem lub symulatorem. Konfiguracja kontrolerów ROS 2 jest odrębna od kontrolerów wykonawczych MoveIt 2 omówionych wcześniej — dotyczy ona bezpośrednio warstwy niskopoziomowej komunikacji ze sprzętem.

Asystent umożliwia automatyczne wygenerowanie konfiguracji dla standardowego typu `joint_trajectory_controller/JointTrajectoryController`. Kontroler ten służy do sterowania ruchem manipulatora poprzez zadawanie ciągłych trajektorii dla jego przegubów, otrzymywanych z warstwy planowania MoveIt 2.

Na rysunku 5.5 widoczna jest konfiguracja kontrolera `arm_trajectory_controller` wraz z pełną listą przegubów, którymi zarządza, oraz parametrami czasowymi sterowania.



Rysunek 5.5: Interfejs Asystenta MoveIt 2: Konfiguracja interfejsu dla sterowników `ros2_control`

5.7. Generowanie pakietu konfiguracyjnego

Po przejściu przez wszystkie powyższe etapy Asystent MoveIt 2 generuje kompletny pakiet ROS 2 zawierający:

- pliki **launch** — skrypty uruchamiające węzły MoveIt 2, serwery parametrów oraz komponenty symulacji,
- pliki konfiguracyjne **YAML** — szczegółowe parametry grup planowania, kontrolerów i limitów kinematycznych,
- pliki **SRDF** (Semantic Robot Description Format) — semantyczną opisującą grupy przegubów, kollisiony i właściwości robota,
- skrypty **Python/C++** — interfejsy programistyczne do interakcji z węzłami MoveIt 2.

Wszystkie te komponenty stanowią kompletną konfigurację niezbędną do uruchomienia systemu planowania ruchu dla opisywanego manipulatora w środowisku ROS 2.

5.8. Podsumowanie konfiguracji

Proces konfiguracji przeprowadzony przy użyciu Asystenta MoveIt 2 umożliwił proste i zautomatyzowane przygotowanie wszystkich parametrów niezbędnych do poprawnego wygenerowania pakietu zawierającego środowisko symulacyjne oraz planer ruchu. Otrzymany pakiet stanowi solidną podstawę do dalszych etapów pracy, pozwalając na łatwe testowanie algorytmów planowania, weryfikację kinematyki, integrację z symulacją oraz współpracę z innymi pakietami i węzłami systemu ROS 2.

6. Funkcjonalności pakietu MoveIt2

6.1. Możliwości kinematyczne manipulatora

Pakiet MoveIt 2 [18] umożliwia realizację zaawansowanych operacji kinematycznych manipulatora, w tym:

- przesunięcie efektora na zadaną pozycję w przestrzeni roboczej (translacja),
- zmianę orientacji efektora względem ustalonego układu współrzędnych (rotacja),
- planowanie trajektorii wolnych od kolizji między konfiguracje początkową i końcową,
- weryfikację osiągalności żądanego położenia w granicach przestrzeni konfiguracyjnej manipulatora,
- detekcję i wizualizację kolizji z przeszkodami oraz samego manipulatora.

Wszystkie wymienione funkcjonalności są dostępne za pośrednictwem interfejsu wizualnego MoveIt Motion Planning w środowisku RViz, stanowiącego narzędzie do interaktywnego sterowania i testowania manipulatora.

6.2. Konwencje wizualizacji w środowisku MoveIt

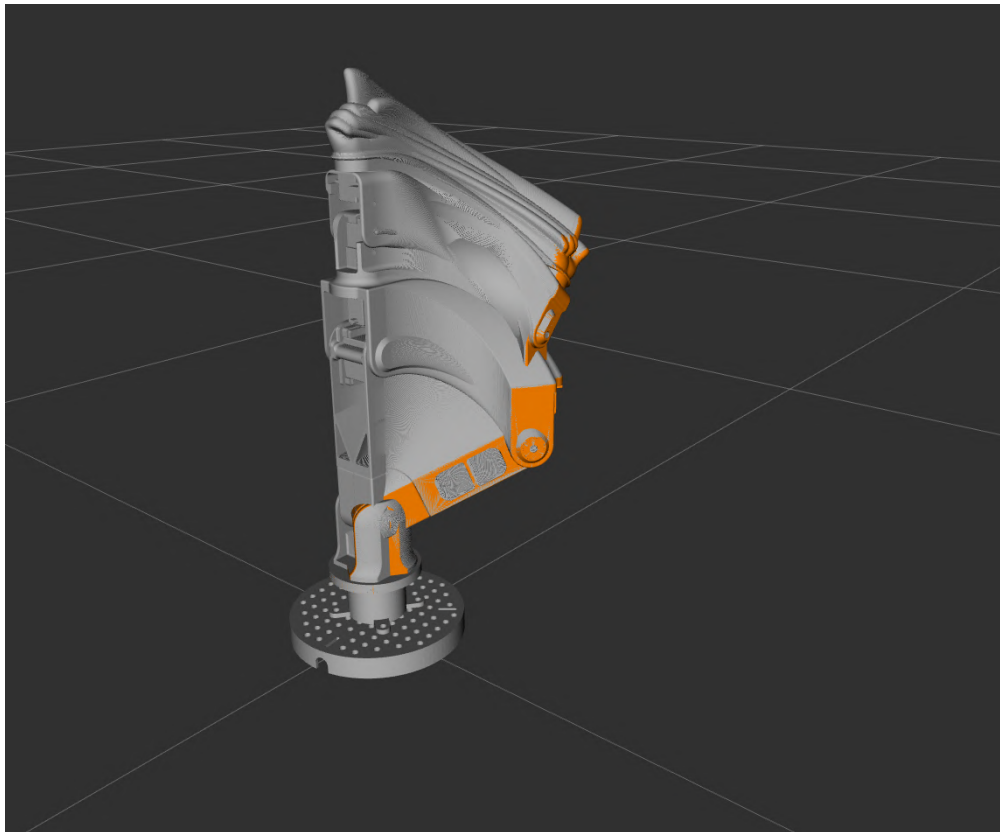
Podczas pracy w interfejsie MoveIt 2 poszczególne stany manipulatora reprezentowane są następującymi kolorami:

- **szary** — aktualna pozycja modelu robota, odzwierciedlająca rzeczywisty (lub początkowy) stan konfiguracyjny,
- **pomarańczowy** — docelowa pozycja, do której ma zmierzyć manipulator w wyniku wykonania zaplanowanej trajektorii,
- **czerwony** — oznacza człony będące w stanie kolizji lub narażone na kolizję.

6.3. Planowanie i weryfikacja trajektorii

Jedną z kluczowych funkcjonalności MoveIt2 jest możliwość **zaplanowania ruchu** z zadanej pozycji początkowej do żądanej pozycji końcowej. Po wyznaczeniu trajektorii system wyświetla graficzną reprezentację planowanej ścieżki, pokazując kolejne konfiguracje, przez które przejdzie manipulator.

Na rysunku 6.1 zaprezentowano przykład zaplanowanej trajektorii. Na wizualizacji narysowane są wszystkie pośrednie konfiguracje, w których znalazł się manipulator. Reprezentuje to ścieżkę przejścia między dwoma punktami referencyjnymi - początkowym i końcowym.

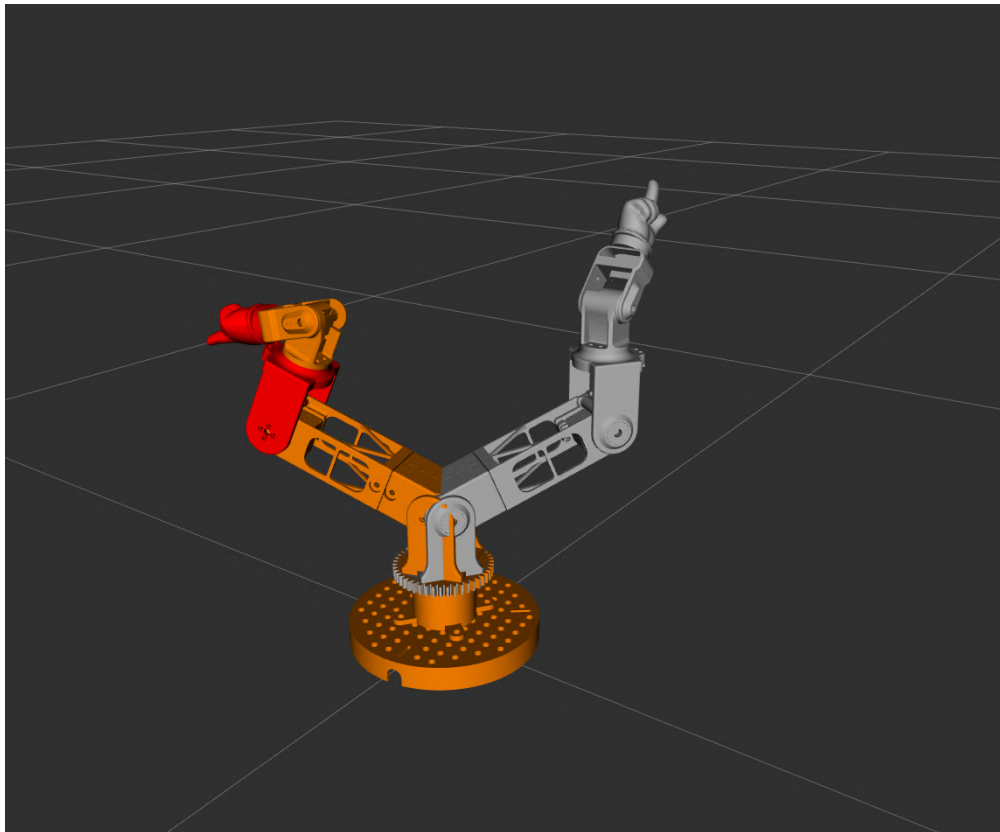


Rysunek 6.1: Przykład zaplanowanej trajektorii manipulatora w środowisku MoveIt 2

6.4. Detekcja kolizji

Ważnym aspektem planowania bezpiecznego ruchu jest **detekcja kolizji**. MoveIt2 automatycznie sprawdza, czy zaplanowana trajektoria lub docelowa pozycja końcowa powoduje kolizję manipulatora z przeszkodami w otoczeniu lub samym sobą (autokolizja).

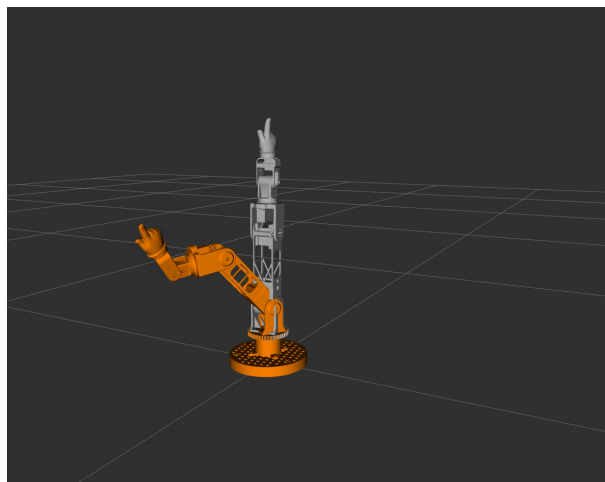
Jeśli robot zostanie umieszczony w stanie kolizji, poszczególne człony będące w konflikcie geometrycznym podświetlają się na **kolor czerwony**. Na rysunku 6.2 pokazano stan, w którym dwa człony manipulatora są w kolizji. Są one zaznaczone kolorem czerwonym, sygnalizując niemożność realizacji takiej konfiguracji.



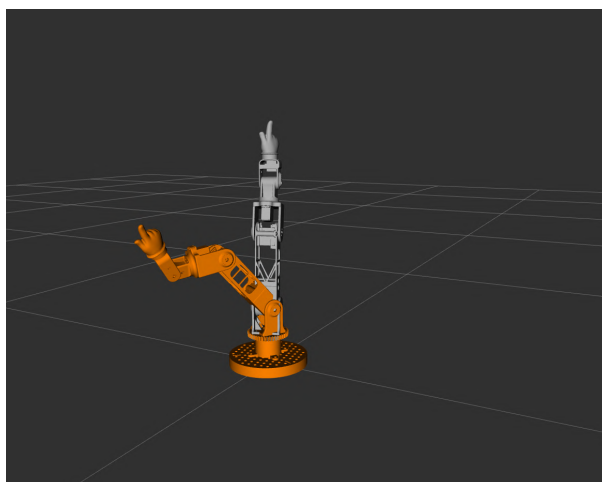
Rysunek 6.2: Detekcja kolizji — człony w konflikcie geometrycznym zaznaczone kolorem czerwonym

6.5. Analiza pośrednich konfiguracji trajektorii

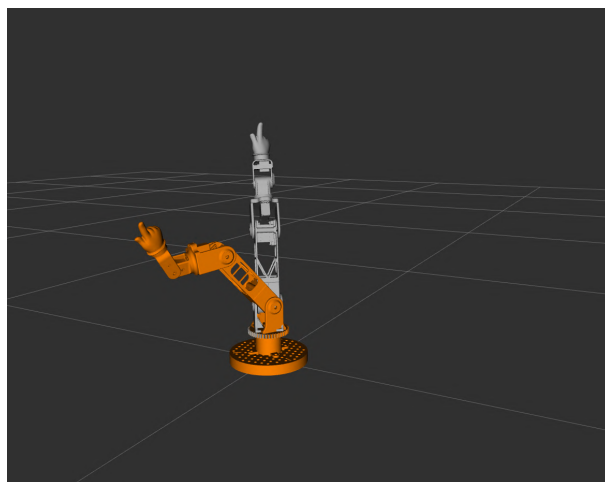
Pakiet MoveIt2 umożliwia szczegółową analizę pośrednich konfiguracji manipulatora w trakcie wykonywania zaplanowanej trajektorii. Funkcjonalność ta jest nieoceniona do weryfikacji, czy ruch przebiega bezpiecznie i zgodnie z oczekiwaniami. Poniżej pokazano sekwencję fotografii stanu manipulatora w kolejnych pośrednich pozycjach, podczas realizacji przykładowego ruchu.



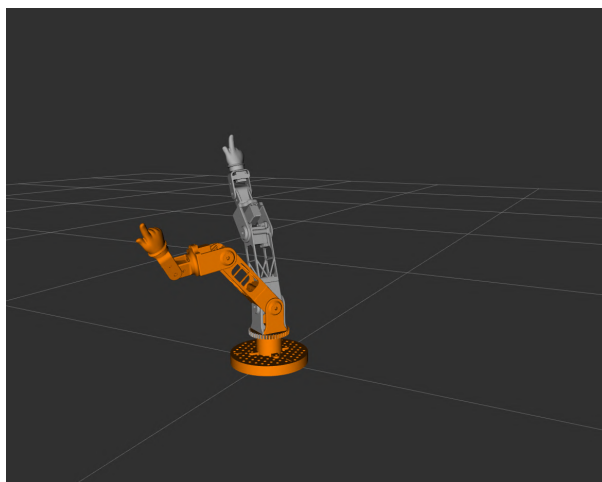
(a) Konfiguracja początkowa



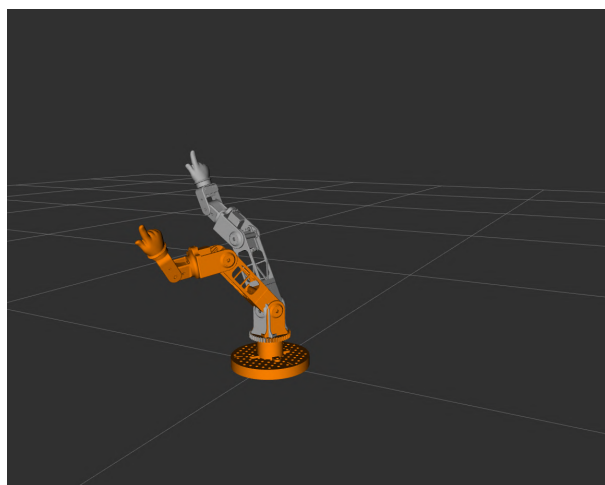
(b) Pierwszy etap



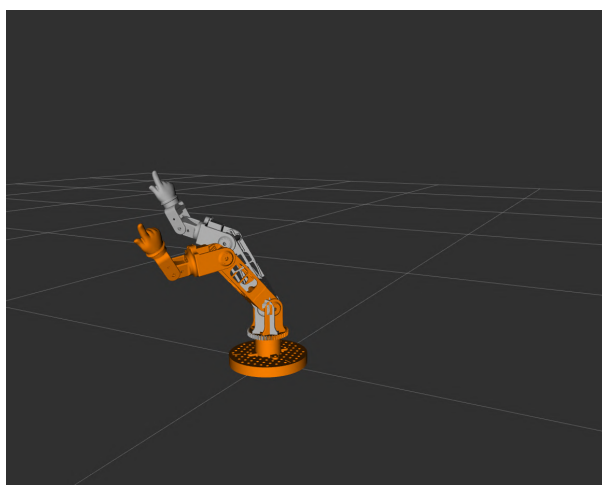
(c) Drugi etap



(d) Trzeci etap



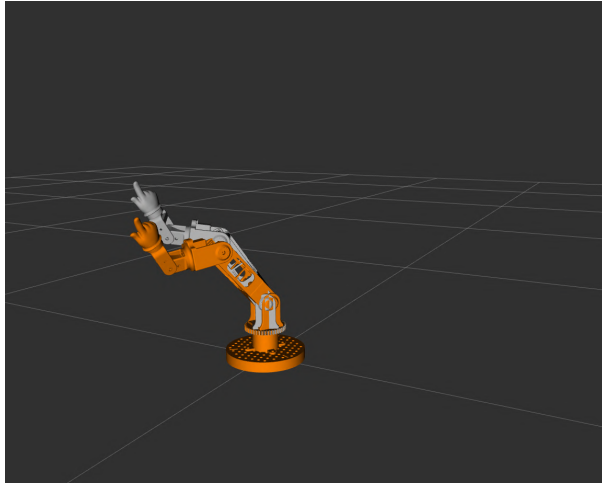
(e) Czwarty etap



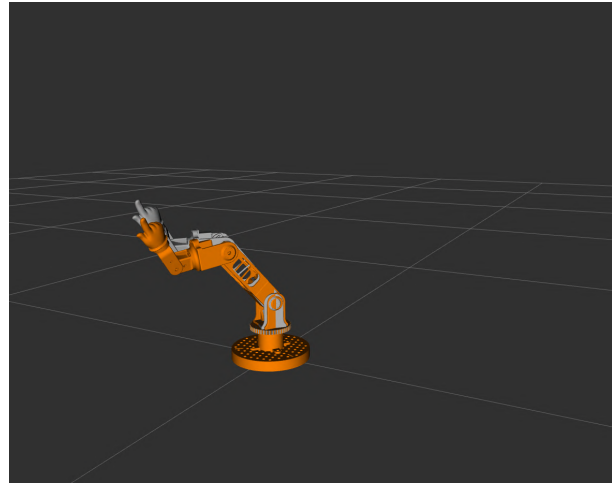
(f) Piąty etap

Rysunek 6.3: Sekwencja sześciu etapów zaplanowanej trajektorii manipulatora (część I)

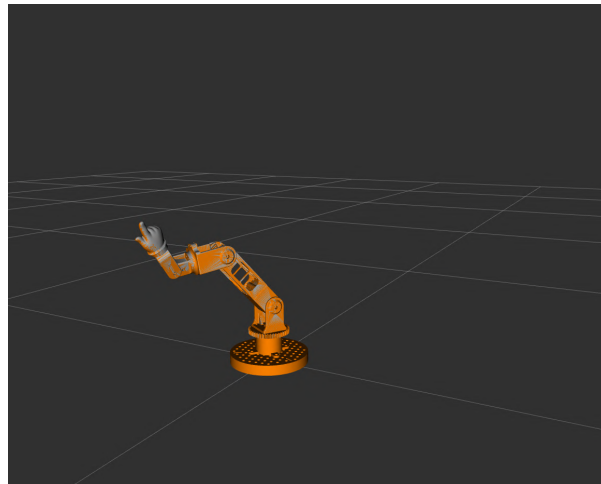
Kontynuacja sekwencji trajektorii — ostatnie trzy etapy przed osiągnięciem pozycji finalnej:



(g) Szósty etap



(h) Siódmy etap



(j) Pozycja końcowa

Rysunek 6.4: Sekwencja ostatnich trzech etapów zaplanowanej trajektorii manipulatora (część II)

Z powyższych ilustracji (rysunki 6.3 i 6.4) wynika, że zaplanowana trajektoria przebiega płynnie, a manipulator przechodzi przez kolejne konfiguracje pośrednie bez kolizji, osiągając żadaną pozycję finalną efektora.

6.6. Podsumowanie

Opisane funkcjonalności pakietu MoveIt2 (planowanie trajektorii, detekcja kolizji oraz interaktywna wizualizacja) stanowią intuicyjne narzędzie do weryfikacji kinematyki manipulatora oraz walidacji bezpieczeństwa zaplanowanych ruchów. Możliwość szczegółowej analizy pośrednich konfiguracji w czasie rzeczywistym. Umożliwia to identyfikację potencjalnych problemów na etapie planowania, przed faktycznym wykonaniem ruchu na rzeczywistym urządzeniu, co pozwala znaczne zmniejszenie ryzyka awarii i kolizji.

7. Sterownik manipulatora

7.1. Architektura systemu sterowania

Niskopoziomowe sterowanie manipulatorem realizowane jest za pomocą mikrokontrolera ESP32-S3 N16R8 [19], który pełni rolę sterownika odpowiedzialnego za bezpośrednią komunikację z serwomechanizmami oraz przetwarzanie sygnałów sprzężenia zwrotnego. Sterownik odbiera polecenia z warstwy wyższej ROS 2 poprzez komunikację szeregową UART [20, 21] i wykonuje rzeczywiste ruchy manipulatora.

Oprogramowanie sterownika zbudowane jest w oparciu o środowisko PlatformIO [22]. Kod źródłowy składa się z modułów odpowiedzialnych za obsługę serw, komunikację oraz kontrolę pętli sterowania.

7.2. Główna pętla sterowania

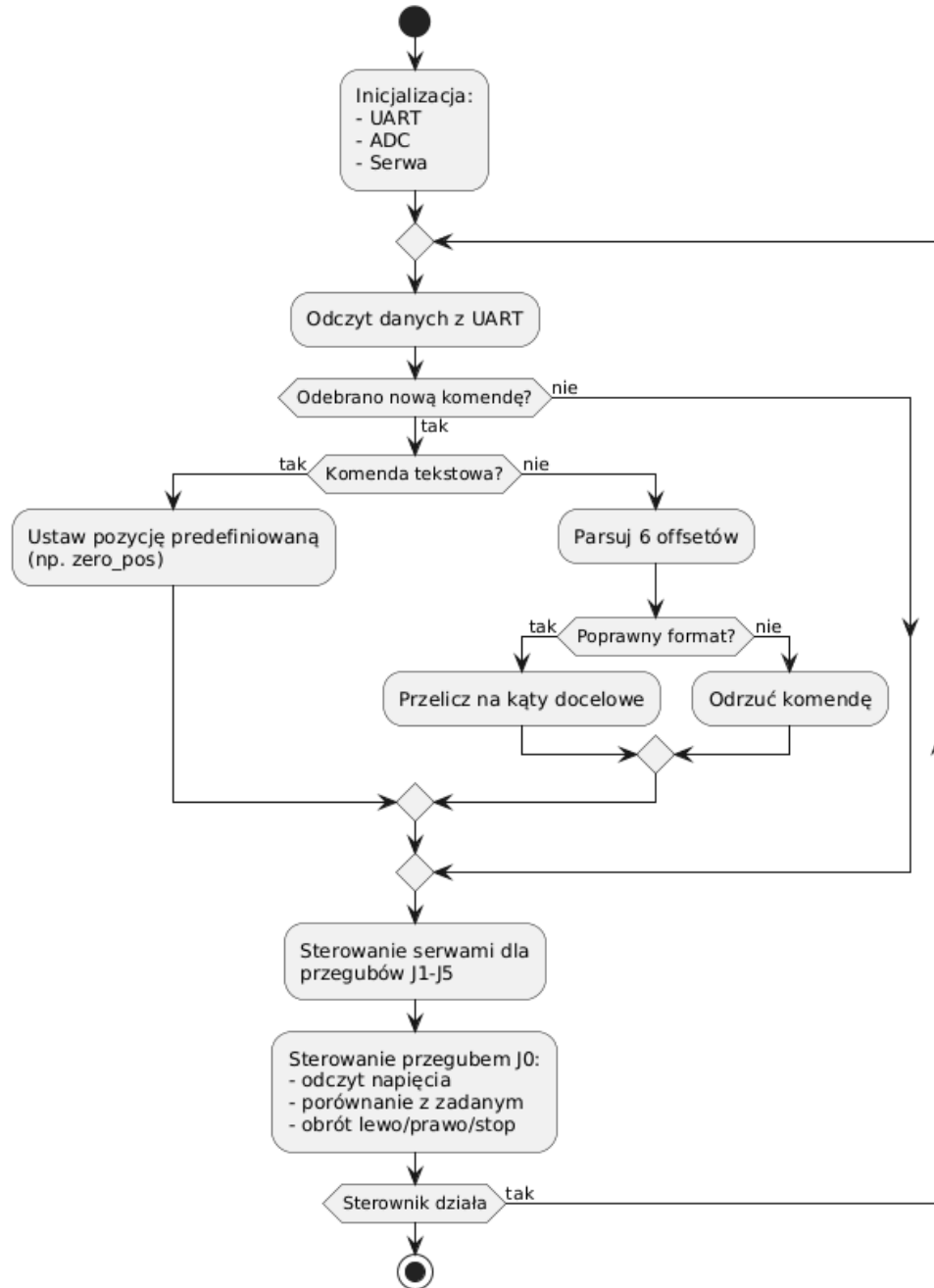
Sercem systemu jest pętla sterowania realizowana przez funkcje `setup()` i `loop()`, które stanowią standardowy interfejs Arduino dla ESP32. Na rysunku 7.1 pokazano schemat działania głównej pętli sterowania.

Funkcja `setup()` realizuje inicjalizację systemu:

- inicjalizacja komunikacji szeregowej UART z prędkością **115200 bauds**,
- konfiguracja rozdzielczości przetwornika ADC na **12 bitów**,
- inicjalizacja wszystkich serwomechanizmów oraz alokacja timerów do sygnału PWM,
- ustawienie początkowych pozycji serwomechanizmów.

Funkcja `loop()` wykonywana jest cyklicznie i realizuje trzy główne zadania:

- **przetwarzanie wejścia szeregowego** (`processSerialInput()`) — odbieranie poleceń z komputera hosta,
- **aktualizacja sprzężenia zwrotnego dla przegłubu 0** (`updateJoint0_feedback()`) — sterowanie regulacyjne przegubu z potencjometra zwrotnego,
- **aktualizacja serwomechanizmów standardowych** (`updateStandardServosSmooth()`) — płynna zmiana pozycji przegubów 1–5.

Schemat blokowy działania sterownika

Rysunek 7.1: Schemat działania sterownika manipulatora

7.3. Komunikacja szeregową i wykonywanie poleceń

Sterownik ESP32 komunikuje się z warstwą wyższą ROS 2 poprzez interfejs UART, umożliwiając wysyłanie poleceń dotyczących pozycji serwomechanizmów. Moduł **SerialCommands** odpowiada za odbieranie, parsowanie i wykonywanie poleceń.

Obsługiwane polecenia obejmują:

- **default** — przejście do predefiniowanej pozycji domyślnej,
- **pos1, pos2, pos3** — przejście do predefiniowanych pozycji (1, 2, 3),
- **zero_pos** — ustawienie neutralnej konfiguracji (wszystkie przeguby w pozycji zerowej),
- **polecenia z MoveIt 2** — sześć wartości liczbowych rozdzielonych spacją.

Polecenia wysyłane z środowiska symulacyjnego mają postać: *offset_J0 offset_J1 offset_J2 offset_J3 offset_J4 offset_J5*, gdzie każdy offset reprezentuje zmianę kąta względem pozycji zerowej, przeskalowaną za pomocą współczynników **offsetMultipliers**. Przykładowa komenda: `30 0 -10 0 0 0` spowoduje obrót przegubu 0 o $+30^\circ$ oraz przegubu 2 o -10° względem pozycji **zero_pos**.

7.4. Sterowanie serwomechanizmami

Sterowanie serwomechanizmami realizowane jest poprzez modulację szerokości impulsu (PWM) zgodnie z zależnością:
gdzie:

- t_{PWM} – czas trwania impulsu wysokiego stanu w mikrosekundach,
- θ – docelowa pozycja kątowa przegubu [rad],
- $\theta_{min}, \theta_{max}$ – mechaniczne ograniczenia zakresu ruchu przegubu,
- $t_{min} = 500\mu s, t_{max} = 2500\mu s$ – parametry serwonapędu według producenta.

8. Integracja sterownika z ROS 2

8.1. Przepływ danych w systemie

Integracja środowiska planowania ruchu (MoveIt 2) z fizycznym sterownikiem została zrealizowana poprzez węzeł ROS 2, który odbiera bieżące wartości przegubów i przesyła je magistralą UART do mikrokontrolera. Przepływ danych przebiega następująco:

1. Moduł planowania ruchu MoveIt 2 wyznacza trajektorię w przestrzeni konfiguracyjnej.
2. W czasie wykonywania ruchu publikowane są stany przegubów w wiadomościach `sensor_msgs/msg/JointState` na temacie `/joint_states`.
3. Węzeł `joint_sender_node` subskrybuje `/joint_states` i odczytuje wektor `position` zawierający pozycje przegubów (w radianach).
4. Odebrane wartości są przeliczane na stopnie, zaokrąglane do liczb całkowitych i porównywane z ostatnio przesłanym stanem.
5. Jeżeli dla dowolnego przegubu zmiana wynosi co najmniej 1 stopień, węzeł wysyła pełny zestaw pozycji przegubów przez UART w postaci jednej linii tekstu.

8.2. Specyfikacja węzła komunikacyjnego

Węzeł pośredniczący (ROS 2 $\rightarrow$ UART) został zaimplementowany w języku C++ z użyciem biblioteki `rclcpp`. Jego zadania obejmują:

- subskrypcję tematu `/joint_states` (typ `sensor_msgs/msg/JointState`),
- konwersję pozycji przegubów z radianów na stopnie oraz ich kwantyzację (zaokrąglenie),
- detekcję istotnej zmiany pozycji (próg $\geq 1^\circ$ względem ostatnio wysłanego stanu),
- formatowanie i transmisję danych do urządzenia szeregowego `/dev/ttyACM0` z prędkością 115200 bitów na sekundę.

8.3. Opis algorytmu

Algorytm w węźle komunikacyjnym odpowiada za ograniczenie liczby transmisji poprzez wysyłanie danych tylko wtedy, gdy zmiana pozycji jest zauważalna z punktu widzenia sterowania. Po odebraniu wiadomości `JointState` wykonywane są następujące kroki:

1. Odczyt wektora `position` zawierającego pozycje przegubów w radianach.
2. Przeliczenie każdej pozycji na stopnie zgodnie z zależnością:

$$\theta_{deg} = \theta_{rad} \cdot \frac{180}{\pi}$$

oraz zaokrąglenie do liczby całkowitej.

3. Sprawdzenie warunku wysyłki:

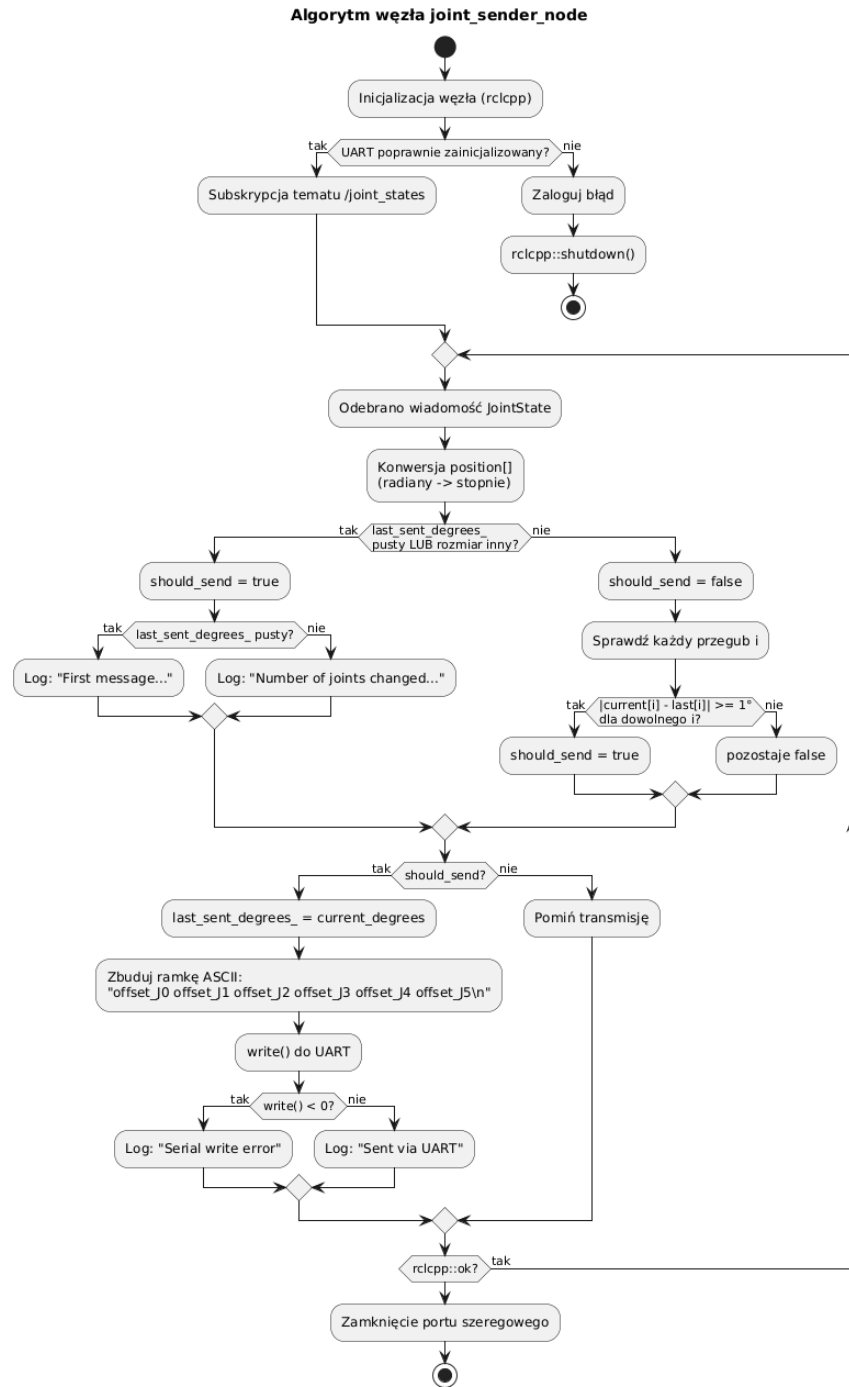
- Jeśli jest to pierwsza odebrana wiadomość, dane są wysyłane natychmiast.
- Jeśli liczba przegubów uległa zmianie (rozmiar wektora), wysyłany jest zaktualizowany stan.
- W przeciwnym przypadku dane są wysyłane tylko wtedy, gdy dla dowolnego przegubu zachodzi:

$$|\theta_{deg}^{(i)} - \theta_{last}^{(i)}| \geq 1.$$

4. W przypadku spełnienia warunku transmisji, wektor `last_sent_degrees` jest aktualizowany, a następnie generowana jest ramka tekstowa i przesyłana przez UART.

Takie podejście redukuje obciążenie łącza UART oraz ogranicza liczbę niepotrzebnych aktualizacji po stronie mikrokontrolera, zachowując jednocześnie spójność ruchu dla typowych trajektorii.

W celu czytelnego przedstawienia logiki działania węzła komunikacyjnego opracowano diagram aktywności, ilustrujący kolejne etapy przetwarzania wiadomości typu `JointState`. Diagram ten przedstawiono na rysunku 8.1.



Rysunek 8.1: Diagram aktywności algorytmu działania węzła joint_sender_node

8.4. Metodologia testowania

Walidację integracji fizycznego manipulatora z symulacją przeprowadzono w dwóch etapach: testy komunikacji oraz testy zachowania układu podczas wykonywania trajektorii. Zastosowano metodę obserwacyjną oraz porównanie zachowania modelu w symulacji i na stanowisku rzeczywistym.

- **Testy komunikacji UART** - weryfikacja poprawności konfiguracji portu oraz poprawności formatu ramek.
- **Testy zgodności ruchów** - ocena zgodności jakościowej: płynności, braku „szarpnięć” wynikających z kwantyzacji w stopniach oraz stabilności pozycji końcowej po zakończeniu ruchu.

8.5. Testy realizacji ruchu

W celu weryfikacji poprawności działania całego łańcucha sterowania (MoveIt 2 → ROS 2 → UART → ESP32) przeprowadzono testy polegające na realizacji zaplanowanych trajektorii ruchu manipulatora pomiędzy pozycją początkową i końcową.

W trakcie testów analizowano zgodność ruchu manipulatora fizycznego z trajektorią zaplanowaną w środowisku symulacyjnym. W wizualizacji środowiska MoveIt 2 kolor **pomarańczowy** oznaczał pozycję docelową efektora, natomiast kolor **szary** przedstawiał aktualną konfigurację manipulatora w trakcie realizacji ruchu.

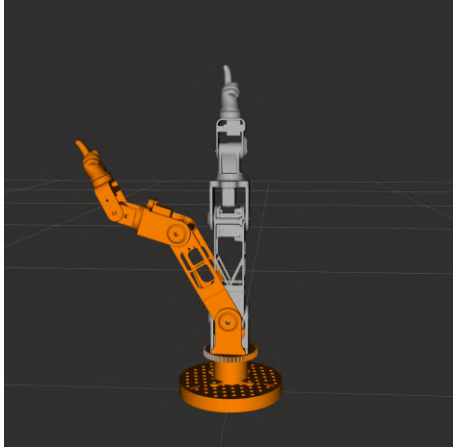
Na podstawie przeprowadzonych testów zaobserwowano:

- płynną realizację zaplanowanych trajektorii w czasie rzeczywistym,
- brak zauważalnych opóźnień pomiędzy symulacją a ruchem manipulatora fizycznego,
- poprawne odwzorowanie końcowych konfiguracji przegubów,
- brak nieprzewidzianych ruchów, drgań oraz kolizji w trakcie ruchu,
- ograniczenie liczby transmisji UART dzięki zastosowaniu progu 1° bez negatywnego wpływu na ciągłość i dokładność ruchu.

Manipulator fizyczny poruszał się w sposób analogiczny do modelu symulacyjnego, a jego konfiguracje w kolejnych punktach trajektorii odpowiadały wartościom zadany przez planer ruchu MoveIt 2. Potwierdza to poprawną synchronizację pomiędzy warstwą planowania, komunikacji oraz sterowania sprzętowego.

Przykładową realizację trajektorii testowej przedstawiono na rysunkach 8.2–8.4. Dla każdej pozycji pokazano zestawienie konfiguracji manipulatora uzyskanej w środowisku symulacyjnym oraz odpowiadającej jej konfiguracji manipulatora fizycznego.

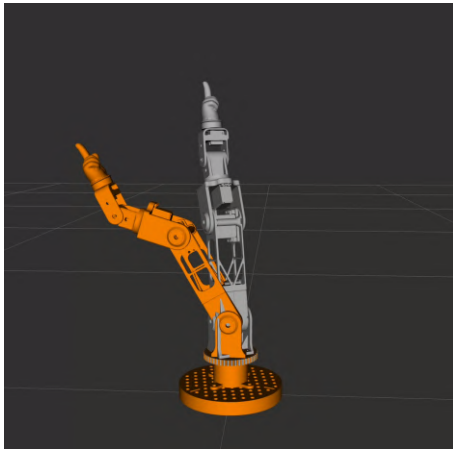
W wizualizacji symulacyjnej kolor **pomarańczowy** oznacza pozycję docelową efektora, natomiast kolor **szary** przedstawia aktualną konfigurację manipulatora w trakcie realizacji ruchu.



(a) Symulacja – pozycja 1



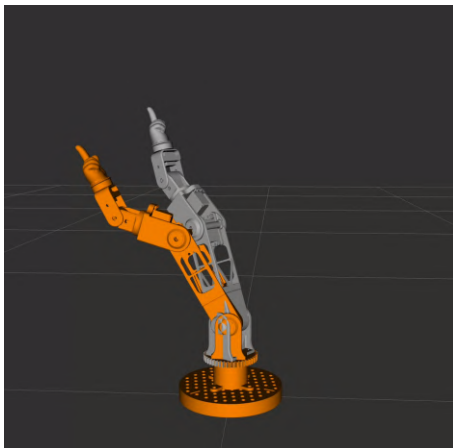
(b) Rzeczywisty manipulator – pozycja 1



(c) Symulacja – pozycja 2



(d) Rzeczywisty manipulator – pozycja 2

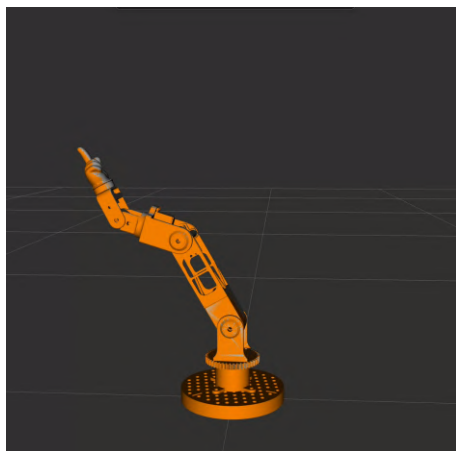


(e) Symulacja – pozycja 3



(f) Rzeczywisty manipulator – pozycja 3

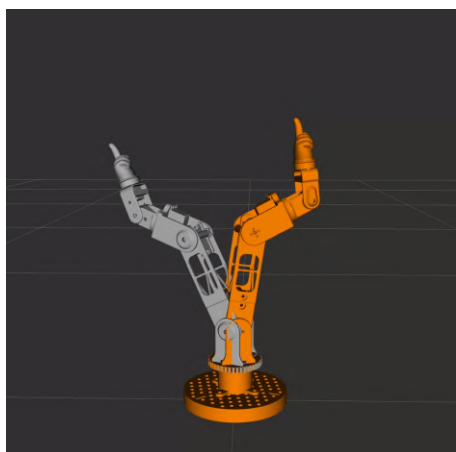
Rysunek 8.2: Porównanie realizacji trajektorii dla pozycji 1–3



(a) Symulacja – pozycja 4



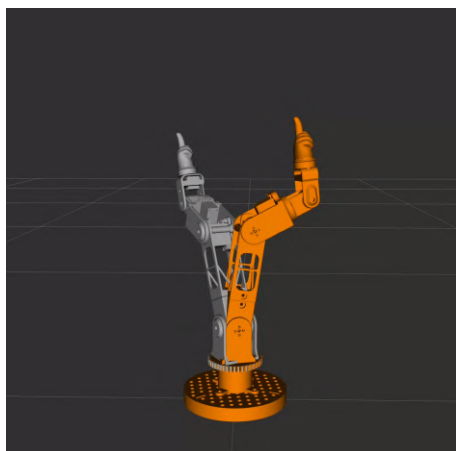
(b) Rzeczywisty manipulator – pozycja 4



(c) Symulacja – pozycja 5



(d) Rzeczywisty manipulator – pozycja 5

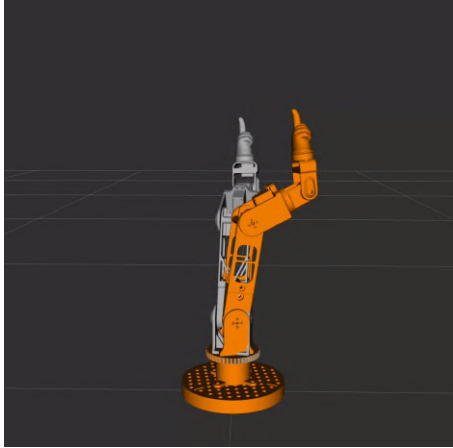


(e) Symulacja – pozycja 6



(f) Rzeczywisty manipulator – pozycja 6

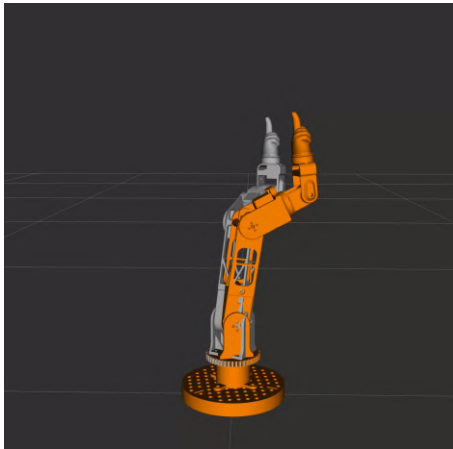
Rysunek 8.3: Porównanie realizacji trajektorii dla pozycji 4–6



(a) Symulacja – pozycja 7



(b) Rzeczywisty manipulator – pozycja 7



(c) Symulacja – pozycja 8



(d) Rzeczywisty manipulator – pozycja 8



(e) Symulacja – pozycja 9



(f) Rzeczywisty manipulator – pozycja 9

Rysunek 8.4: Porównanie realizacji trajektorii dla pozycji 7–9

9. Wnioski i prace przyszłe

9.1. Podsumowanie

Celem niniejszej pracy było zaprojektowanie oraz wykonanie sześciostopniowego manipulatora robotycznego wraz z jego integracją ze środowiskiem ROS 2, umożliwiającą planowanie ruchu w przestrzeni konfiguracyjnej oraz realizację trajektorii na rzeczywistym obiekcie. Postawiony cel został osiągnięty poprzez realizację kompletnego procesu inżynierskiego, obejmującego zarówno projekt mechaniczny, analizę kinematyczną, implementację systemu sterowania oraz komunikacji pomiędzy warstwą symulacyjną a sterownikiem.

W ramach pracy opracowano model manipulatora w środowisku CAD, który następnie został wykonany z wykorzystaniem technologii druku 3D. Na jego podstawie przygotowano cyfrowy odpowiednik robota w formacie URDF/XACRO, umożliwiający poprawną wizualizację oraz analizę kinematyki w środowisku ROS 2. Stworzona konfiguracja pakietu MoveIt 2 pozwoliła na planowanie bezkolizyjnych trajektorii, ich weryfikację w symulacji oraz interaktywną analizę zachowania manipulatora.

Istotnym elementem pracy było zaprojektowanie i implementacja niskopoziomowego sterownika opartego na mikrokontrolerze ESP32-S3. Opracowane oprogramowanie umożliwia sterowanie serwomechanizmami manipulatora oraz realizację ruchów zadanych z poziomu środowiska ROS 2. Zaimplementowany węzeł komunikacyjny zapewnia jednokierunkową transmisję danych pomiędzy symulacją a układem wbudowanym. Pozwoliło to na zsynchronizowanie ruchu modelu wirtualnego z rzeczywistym manipulatorem.

Przeprowadzone testy potwierdziły poprawność działania całego systemu oraz zgodność ruchów wykonywanych przez fizyczny manipulator z trajektoriami planowanymi w środowisku MoveIt 2.

9.2. Ograniczenia zrealizowanego systemu

Pomimo poprawnej realizacji założeń projektowych, w trakcie prac oraz testów systemu zidentyfikowano pewne ograniczenia wynikające głównie z przyjętych kompromisów konstrukcyjnych oraz zastosowanych komponentów. Najważniejszym z nich jest ograniczona dokładność pozycjonowania, wynikająca z charakterystyki użytych serwomechanizmów. Zauważalne luzy przekładni oraz ograniczona rozdzielczość zadawania pozycji wpływają na precyzję odwzorowania zaplanowanych trajektorii.

Dodatkowym ograniczeniem jest brak pełnego sprzężenia zwrotnego dla wszystkich przegubów manipulatora. Jednokierunkowa komunikacja pomiędzy środowiskiem ROS 2 a sterownikiem ESP32 uniemożliwia bieżącą weryfikację rzeczywistego stanu manipulatora, a tym samym detekcję błędów wykonania ruchu, kolizji czy rozbieżności pomiędzy modelem cyfrowym a fizycznym. Ograniczenie to wpływa również na możliwość implementacji bardziej zaawansowanych algorytmów regulacji czy kompensacji błędów.

9.3. Kierunki dalszego rozwoju

Opracowany system stanowi solidną bazę do dalszych prac rozwojowych i badawczych. Jednym z kluczowych kierunków rozwoju jest implementacja pełnego sprzężenia zwrotnego, opartego na czujnikach położenia lub enkoderach dla wszystkich przegubów. Umożliwiłoby to bieżącą synchronizację stanu manipulatora z modelem w środowisku ROS 2 oraz zastosowanie algorytmów regulacji zamkniętej pętli.

Istotnym rozszerzeniem funkcjonalnym byłoby również wprowadzenie komunikacji bezprzewodowej pomiędzy manipulatorem a systemem nadrzędnym. Zastosowanie interfejsów takich jak Wi-Fi lub Bluetooth, dostępnych natywnie w mikrokontrolerze ESP32, pozwoliłoby na zwiększenie mobilności systemu, redukcję okablowania oraz łatwiejszą integrację.

Kolejnym możliwym kierunkiem rozwoju jest rozbudowa interfejsu użytkownika, umożliwiającą ręczne prowadzenie manipulatora lub zadawanie trajektorii w czasie rzeczywistym, na przykład z wykorzystaniem panelu operatorskiego typu *teach pendant*.

Zaproponowane kierunki rozwoju pokazują, że wykonany manipulator może stanowić punkt wyjścia do dalszych badań oraz kolejnych iteracji projektowych.

Spis rysunków

2.1	Rysunek poglądowy członu bazowego (człon 0)	9
2.2	Człon nr 2 manipulatora	10
2.3	Człon nr 2 manipulatora	11
2.4	Człon nr 3 manipulatora	12
2.5	Człon nr 4 manipulatora	13
2.6	Człon nr 5 manipulatora	14
2.7	Efektor końcowy manipulatora (człon 6)	15
2.8	Kompletne złożenie manipulatora	16
2.9	Układ sterowania – widok od strony wewnętrznej	17
2.10	Układ sterowania – widok od strony zewnętrznej	17
2.11	Pozycja nr 1	18
2.12	Pozycja pos_zero	18
2.13	Pozycja nr 2	18
2.14	Pozycja nr 3	18
2.15	Rzeczywisty model fizyczny manipulatora w różnych konfiguracjach roboczych	18
3.1	Schemat kinematyczny manipulatora w pozycji zerowej	19
3.2	Model manipulatora w pozycji zerowej	19
4.1	Podgląd modelu manipulatora w Fusion 360 przed eksportem do formatu URDF	28
4.2	Interfejs uruchamiania skryptu eksportu do formatu URDF	29
4.3	Zawartość wygenerowanej paczki	30
4.4	Manipulator w środowisku RViz w pozycji domyślnej	31
4.5	Manipulator w środowisku RViz po rotacji wszystkich przegłębów	32
5.1	Interfejs Asystenta MoveIt 2: Wygenerowana macierz autokolizji (Self-Collision Matrix) dla manipulatora	34
5.2	Interfejs Asystenta MoveIt 2: Definiowanie grup planowania (Planning Groups)	35
5.3	Interfejs Asystenta MoveIt 2: Konfiguracja kontrolerów wykonawczych (MoveIt Controllers)	36
5.4	Interfejs Asystenta MoveIt 2: Definiowanie predefiniowanych póz robota (Robot Poses)	37
5.5	Interfejs Asystenta MoveIt 2: Konfiguracja interfejsu dla sterowników ros2_control	38
6.1	Przykład zaplanowanej trajektorii manipulatora w środowisku MoveIt 2	42
6.2	Detekcja kolizji — człony w konflikcie geometrycznym zaznaczone kolorem czerwonym	43

6.3	Sekwencja sześciu etapów zaplanowanej trajektorii manipulatora (część I) . . .	44
6.4	Sekwencja ostatnich trzech etapów zaplanowanej trajektorii manipulatora (część II)	45
7.1	Schemat działania sterownika manipulatora	48
8.1	Diagram aktywności algorytmu działania węzła <code>joint_sender_node</code>	53
8.2	Porównanie realizacji trajektorii dla pozycji 1–3	55
8.3	Porównanie realizacji trajektorii dla pozycji 4–6	56
8.4	Porównanie realizacji trajektorii dla pozycji 7–9	57

Spis tabel

3.1 Parametry Denavita–Hartenberga manipulatora	20
---	----

Bibliografia

- [1] HowToRobot Editorial Team. Top 12 industrial robot applications and uses, August 2024.
- [2] Robots.com. The many capabilities of an industrial robot, 2023.
- [3] Autodesk. The past, present, and future of industrial robotics, 2023.
- [4] John J. Craig. *Introduction to Robotics: Mechanics and Control*. Pearson Education, 2005.
- [5] KUKA AG. Roboty przemysłowe kuka, 2024.
- [6] ABB Group. Roboty przemysłowe abb, 2024.
- [7] FANUC Europe Corporation. Roboty przemysłowe fanuc, 2024.
- [8] Universal Robots A/S. Roboty przemysłowe universal robots, 2024.
- [9] Botland group. Choosing the right motor - dc vs stepper vs servo. URL: <https://botland.store/content/230-wybor-wlasciwego-silnika-dc-vs-krokowy-vs-serwo>.
- [10] Monolithic Power Systems, Inc. Stepper motors: Basics, types, and uses, 2024.
- [11] Jameco Electronics. How servo motors work, 2023.
- [12] Robert Muszyński Krzysztof Tchoń. Robotyka. Notatki do wykładów z dziedziny automatyki i robotyki, 2018.
- [13] Wikipedia contributors. Denavit–hartenberg parameters, 2024.
- [14] Automatic Addison. Homogeneous transformation matrices using denavit–hartenberg, 2024.
- [15] Wikipedia contributors. Forward kinematics, 2024.
- [16] University of Illinois at Urbana–Champaign. Inverse kinematics, 2024.
- [17] Dheenadhayalan - dheena2k2. fusion2urdf export script for fusion360, 2023.
- [18] PickNik Robotics. Moveit documentation, 2024.
- [19] Espressif Systems. Esp32-s3-wroom-1u-n16r8 datasheet, 2023.
- [20] Analog Devices. Uart: A hardware communication protocol, 2024.
- [21] Espressif Systems. Esp-idf api reference — uart, 2025.
- [22] PlatformIO. Frameworks, 2024.